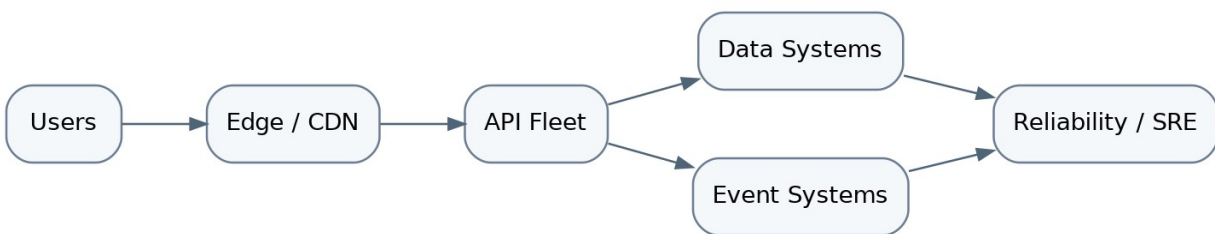


PRACTICAL SYSTEM DESIGN

From a Single HTTP Request to Internet-Scale Systems

A source-backed, visual and implementation-first engineering handbook



Anup Dangi

How to Use This Book

This book is designed for two modes: sequential learning and rapid recall. Each chapter starts from the problem that created the concept, develops a mental model, explains mechanics with a worked example, then forces the architecture through failure scenarios and a small implementation lab. When revising, read the chapter summary, redraw the diagram from memory, and answer the design-review questions without looking.

Recall test

Close the book and explain this page in your own words. A system-design concept is learned only when you can predict how it behaves under load and failure, not when you recognize its diagram.

Source Policy

Mechanics are anchored in primary sources wherever possible: official documentation, original research papers, and engineering articles from the organizations that operated the systems. Alex Xu's System Design Interview is used as grounding for the conceptual progression, while the book links to public primary references beside the topics they support. Real-company diagrams are original educational redraws rather than copied screenshots.

Recall test

Close the book and explain this page in your own words. A system-design concept is learned only when you can predict how it behaves under load and failure, not when you recognize its diagram.

The Senior Engineer Mental Model

For every system ask the same questions: Where does truth live? What is the request path? What is the bottleneck at 10x load? What can be stale? What must never become false? What happens when a dependency disappears? How is overload controlled? What is observable? How is a bad deploy rolled back? What does one successful unit of work cost?

Recall test

Close the book and explain this page in your own words. A system-design concept is learned only when you can predict how it behaves under load and failure, not when you recognize its diagram.

Core Equations

Little's Law: $L = \lambda * W$. Queue growth when overloaded: $\text{backlog_rate} = \lambda - \mu$. Error budget: $1 - \text{SLO}$. Burn rate: $\text{observed error rate} / \text{allowed error rate}$. Quorum overlap heuristic: $R + W > N$. Tail fan-out under independence: $P(\text{all fast}) = p^n$. Risk: $\text{probability of failure} * \text{blast radius}$.

Recall test

Close the book and explain this page in your own words. A system-design concept is learned only when you can predict how it behaves under load and failure, not when you recognize its diagram.

Table of Contents

Part I - Request Path and Web Fundamentals

- Chapter 1. What Happens When You Type a URL?
- Chapter 2. DNS: Naming, Caching, and Failure
- Chapter 3. TCP, QUIC, TLS, and Connection Cost
- Chapter 4. HTTP APIs: Contracts, Idempotency, and Pagination
- Chapter 5. CDNs and Edge Caching
- Chapter 6. Capacity Estimation and Back-of-the-Envelope Math

Part II - Scaling the Backend

- Chapter 7. Vertical vs Horizontal Scaling
- Chapter 8. Load Balancing and Health Checks
- Chapter 9. Stateless Services, Sessions, and Shared State
- Chapter 10. Rate Limiting and Quotas
- Chapter 11. Autoscaling, Saturation, and Warm-Up Time
- Chapter 12. Tail Latency and Fan-Out

Part III - Data Systems

- Chapter 13. Relational Data Modeling and Constraints
- Chapter 14. Transactions and ACID
- Chapter 15. Isolation, MVCC, and Concurrency Anomalies
- Chapter 16. Indexes: B-Tree, Hash, GIN, BRIN
- Chapter 17. Replication, Failover, and Replication Lag
- Chapter 18. Caching: Cache-Aside, TTL, and Stampedes
- Chapter 19. Redis: Persistence, Eviction, and Failure
- Chapter 20. Sharding and Partition-Key Design
- Chapter 21. Consistent Hashing and Membership Churn
- Chapter 22. NoSQL Data Models: KV, Document, Column, Graph
- Chapter 23. Object Storage, Content Addressing, and Large Blobs

Part IV - Distributed and Event-Driven Systems

- Chapter 24. Logical Time, Causality, and Lamport Clocks
- Chapter 25. Quorums and Replicated Reads/Writes
- Chapter 26. Consensus and Raft
- Chapter 27. Distributed IDs and Snowflake
- Chapter 28. Queues, Backpressure, and Queueing Theory
- Chapter 29. Kafka: Logs, Partitions, and Consumer Groups
- Chapter 30. Delivery Semantics and Idempotent Consumers
- Chapter 31. Retry Topics, Poison Messages, and DLQs
- Chapter 32. Transactional Outbox and the Dual-Write Problem
- Chapter 33. Sagas and Distributed Business Transactions
- Chapter 34. Consistency Models and Convergence

Part V - Reliability and Production Operations

- Chapter 35. Observability: Metrics, Logs, Traces, and Baggage

- Chapter 36. SLIs, SLOs, Error Budgets, and Burn Rate
- Chapter 37. Timeouts, Retries, Backoff, and Jitter
- Chapter 38. Circuit Breakers, Bulkheads, and Dependency Isolation
- Chapter 39. Load Shedding and Admission Control
- Chapter 40. Canaries, Feature Flags, and Rollback
- Chapter 41. Multi-Region Design and Disaster Recovery
- Chapter 42. Schema and Data Migrations Without Downtime

Part VI - Architecture Autopsies

- Chapter 43. Discord: Storing Trillions of Messages
- Chapter 44. Discord: Indexing Trillions of Messages
- Chapter 45. GitHub: Partitioning Relational Databases
- Chapter 46. Meta: Scaling Memcache
- Chapter 47. Meta: Haystack Photo Storage
- Chapter 48. Stripe: Rate Limiters as Reliability Infrastructure
- Chapter 49. Uber: Kafka Retry and Dead-Letter Architecture
- Chapter 50. Cloudflare: Durable Objects and Stateful Coordination
- Chapter 51. Cloudflare: Consistent Hashing for Worker Warmth
- Chapter 52. Google: Dapper and Distributed Tracing
- Projects and capstones
- Appendices: design checklist, formulas, failure drills, glossary, source index

PART I - REQUEST PATH AND WEB FUNDAMENTALS

The goal of this part is to build a reusable mental model, not memorize a product stack. Every mechanism is tied to an invariant, a bottleneck, or a failure mode.

Chapter 1 - What Happens When You Type a URL?

Why this exists. A web request is a chain of independent systems. Senior engineers reason about the entire path, because DNS, transport, TLS, proxies, applications, caches, and databases can each add latency or fail independently.

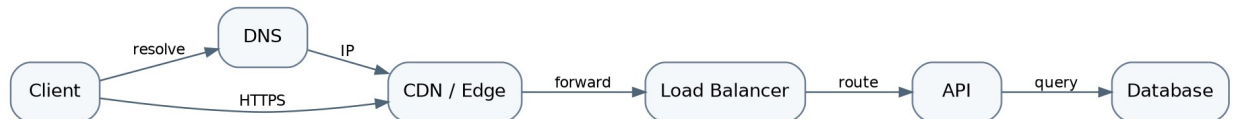


Figure 1.1 - What Happens When You Type a URL?: simplified architecture

Three concepts to retain

- DNS resolves a name to a routable endpoint.
- TCP/QUIC creates transport; TLS establishes identity and encryption.
- HTTP flows through edge, load balancer, application, cache, and database layers.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

What Happens When You Type a URL? - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Latency budget

If DNS takes 30 ms, TCP+TLS 80 ms, edge 20 ms, API 40 ms, and DB 25 ms, the request is already 195 ms before rendering.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If web request path becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

What Happens When You Type a URL? - Failure analysis

Failure scenario

A user in Bengaluru reports a 2-second page while the API dashboard shows 80 ms. Trace every network and browser stage before blaming the application.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

What Happens When You Type a URL? - Build, inspect, and read deeper

Small implementation / lab

Use `curl -v`, `dig`, `traceroute/mtr`, and browser DevTools to document one real request from DNS lookup to response body.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[MDN HTTP overview](https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview) - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

[Cloudflare TLS handshake](https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/) - <https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/>

Recall checklist

- Why does web request path exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 2 - DNS: Naming, Caching, and Failure

Why this exists. DNS is a distributed, cached naming system. Its scalability comes from delegation and caching; its failure modes come from stale caches, misconfiguration, propagation delays, and dependency on resolvers.

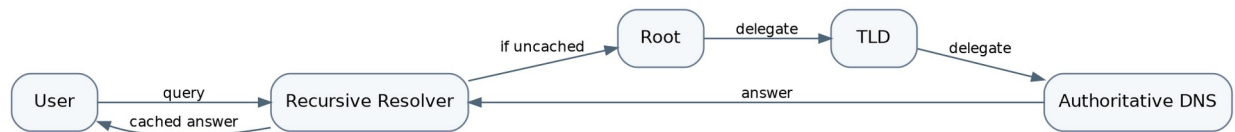


Figure 2.1 - DNS: Naming, Caching, and Failure: simplified architecture

Three concepts to retain

- Recursive resolvers cache answers according to TTL.
- Authoritative servers own zones; registrars and registries form the delegation chain.
- Negative caching and multiple record types affect failover behavior.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

DNS: Naming, Caching, and Failure - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

TTL trade-off

A 300-second TTL allows faster endpoint changes than 24 hours, but causes more resolver traffic.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If DNS becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

DNS: Naming, Caching, and Failure - Failure analysis

Failure scenario

You change an A record during an outage but many clients keep using the old address because the prior TTL was one day.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

DNS: Naming, Caching, and Failure - Build, inspect, and read deeper

Small implementation / lab

Use dig +trace on a domain, inspect NS/A/AAAA/CNAME records, and change a test record with two TTLs to observe propagation.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Cloudflare DNS learning center](https://www.cloudflare.com/learning/dns/what-is-dns/) - <https://www.cloudflare.com/learning/dns/what-is-dns/>

[RFC 1034](https://www.rfc-editor.org/rfc/rfc1034) - <https://www.rfc-editor.org/rfc/rfc1034>

Recall checklist

- Why does DNS exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 3 - TCP, QUIC, TLS, and Connection Cost

Why this exists. Application latency starts below HTTP. Handshakes, congestion control, packet loss, and connection reuse materially affect user-perceived response time.

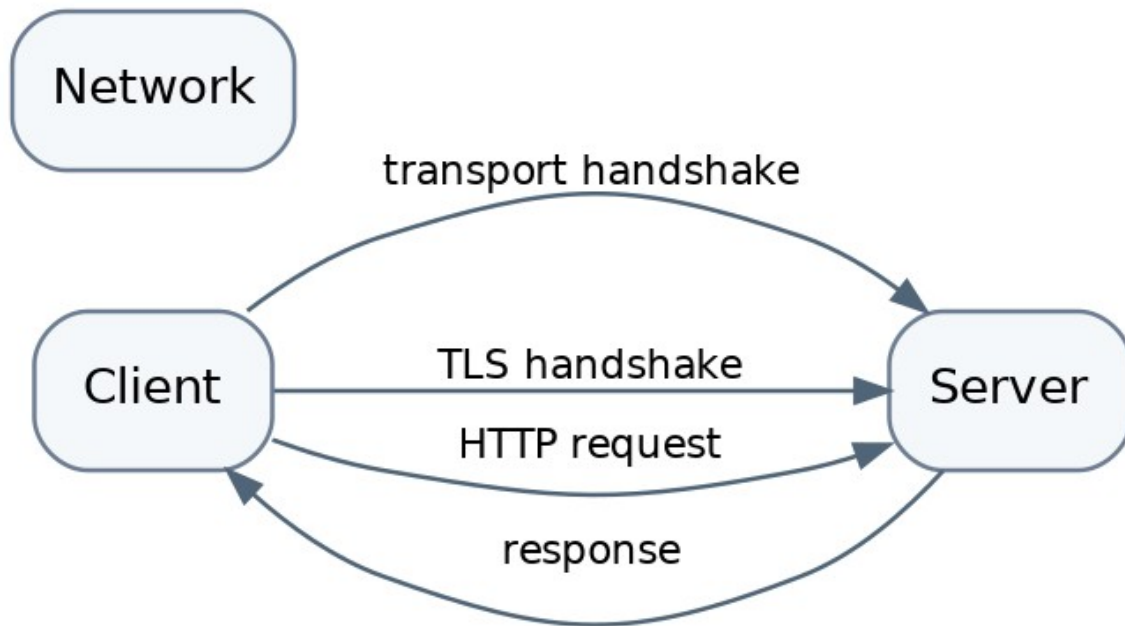


Figure 3.1 - TCP, QUIC, TLS, and Connection Cost: simplified architecture

Three concepts to retain

- TCP requires connection establishment; TLS adds cryptographic negotiation.
- HTTP/2 multiplexes streams over a connection; HTTP/3 uses QUIC over UDP.
- Connection pooling and keep-alive avoid repeated handshake cost.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

TCP, QUIC, TLS, and Connection Cost - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Round trips

On a 120 ms RTT path, two additional round trips cost roughly 240 ms before application work.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If transport security becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

TCP, QUIC, TLS, and Connection Cost - Failure analysis

Failure scenario

A service looks fast in-region but slow globally because every request opens a fresh connection and performs full TLS negotiation.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

TCP, QUIC, TLS, and Connection Cost - Build, inspect, and read deeper

Small implementation / lab

Benchmark the same endpoint with connection reuse enabled and disabled; compare handshake count and p95 latency.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Cloudflare TLS handshake](https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/) - <https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/>

[MDN HTTP/3](https://developer.mozilla.org/en-US/docs/Glossary/HTTP_3) - https://developer.mozilla.org/en-US/docs/Glossary/HTTP_3

Recall checklist

- Why does transport security exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 4 - HTTP APIs: Contracts, Idempotency, and Pagination

Why this exists. An API is a long-lived contract between independently deployed systems. Good API design therefore optimizes for compatibility, retries, observability, and predictable semantics rather than endpoint aesthetics.

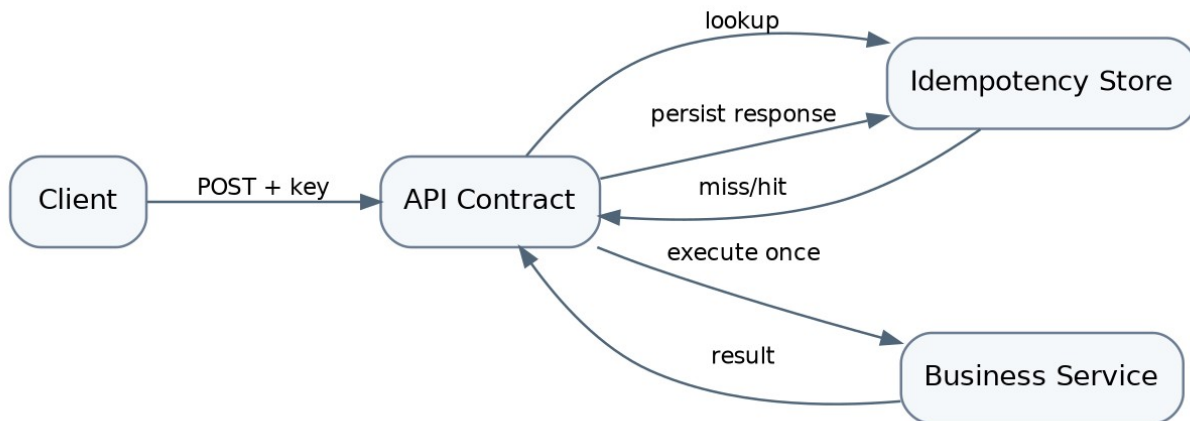


Figure 4.1 - HTTP APIs: Contracts, Idempotency, and Pagination: simplified architecture

Three concepts to retain

- Methods and status codes communicate semantics.
- Idempotency makes retries safe for operations with side effects.
- Pagination, filtering, and versioning bound resource usage and change risk.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

HTTP APIs: Contracts, Idempotency, and Pagination - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Idempotency

If a payment request times out after the charge succeeds, retrying without an idempotency key can charge twice.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If API design becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

HTTP APIs: Contracts, Idempotency, and Pagination - Failure analysis

Failure scenario

Client retries POST /payments after a network timeout; the server executes the same business operation twice.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

HTTP APIs: Contracts, Idempotency, and Pagination - Build, inspect, and read deeper

Small implementation / lab

Implement POST /orders with an idempotency-key table and cursor-based pagination for GET /orders.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Stripe idempotent requests](https://docs.stripe.com/api/idempotent_requests) - https://docs.stripe.com/api/idempotent_requests

[Stripe API versioning](https://stripe.com/blog/api-versioning) - <https://stripe.com/blog/api-versioning>

Recall checklist

- Why does API design exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 5 - CDNs and Edge Caching

Why this exists. A CDN reduces distance, origin load, and tail latency by serving cached content from distributed edge locations. The difficult part is cache correctness and cache-key design.

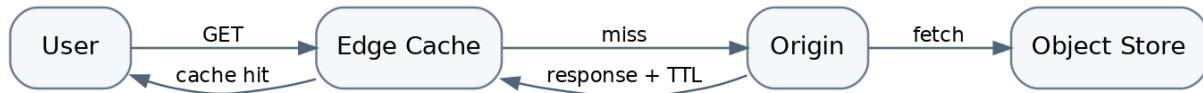


Figure 5.1 - CDNs and Edge Caching: simplified architecture

Three concepts to retain

- Cache keys determine which requests can safely share responses.
- TTL and revalidation balance freshness against origin traffic.
- Purges and invalidation are operationally different from natural expiration.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

CDNs and Edge Caching - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Hit ratio

At 95% cache hit ratio, only 5 of every 100 requests reach the origin.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If CDN becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

CDNs and Edge Caching - Failure analysis

Failure scenario

A cache key ignores the Authorization header and serves one user's private page to another user.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

CDNs and Edge Caching - Build, inspect, and read deeper

Small implementation / lab

Put a static asset behind a CDN; vary Cache-Control and observe Age, ETag, hit/miss behavior, and invalidation.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Cloudflare CDN overview](https://www.cloudflare.com/learning/cdn/what-is-a-cdn/) - <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>

[MDN HTTP caching](https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching) - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching>

Recall checklist

- Why does CDN exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 6 - Capacity Estimation and Back-of-the-Envelope Math

Why this exists. System design gets concrete when requirements become numbers. QPS, storage growth, bandwidth, concurrency, and peak multipliers expose bottlenecks before architecture diagrams do.

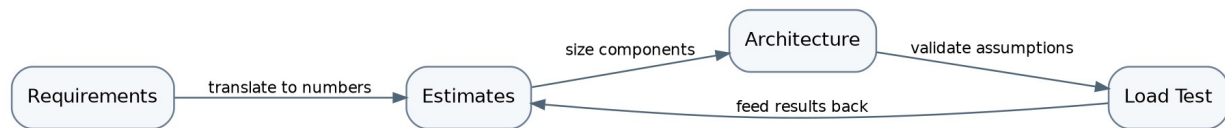


Figure 6.1 - Capacity Estimation and Back-of-the-Envelope Math: simplified architecture

Three concepts to retain

- Average QPS must be converted to realistic peak QPS.
- Storage planning includes indexes, replicas, metadata, and retention.
- Concurrency depends on arrival rate and service time.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Capacity Estimation and Back-of-the-Envelope Math - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Example

10M DAU * 20 requests/day = 200M requests/day = about 2,315 average QPS; with a 5x peak, design around 11,600 QPS.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If capacity planning becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Capacity Estimation and Back-of-the-Envelope Math - Failure analysis

Failure scenario

A design estimates only average traffic and collapses during a two-hour launch peak.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Capacity Estimation and Back-of-the-Envelope Math - Build, inspect, and read deeper

Small implementation / lab

Estimate QPS, egress, storage/year, cache size, DB write rate, and worker count for a chat app with 1M DAU.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Alex Xu companion site](https://bytebytego.com/) - <https://bytebytego.com/>

[AWS Architecture Center](https://aws.amazon.com/architecture/) - <https://aws.amazon.com/architecture/>

Recall checklist

- Why does capacity planning exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

PART II - SCALING THE BACKEND

The goal of this part is to build a reusable mental model, not memorize a product stack. Every mechanism is tied to an invariant, a bottleneck, or a failure mode.

Chapter 7 - Vertical vs Horizontal Scaling

Why this exists. Scaling up buys simplicity; scaling out buys a larger ceiling and fault tolerance at the cost of distribution, coordination, and operational complexity.

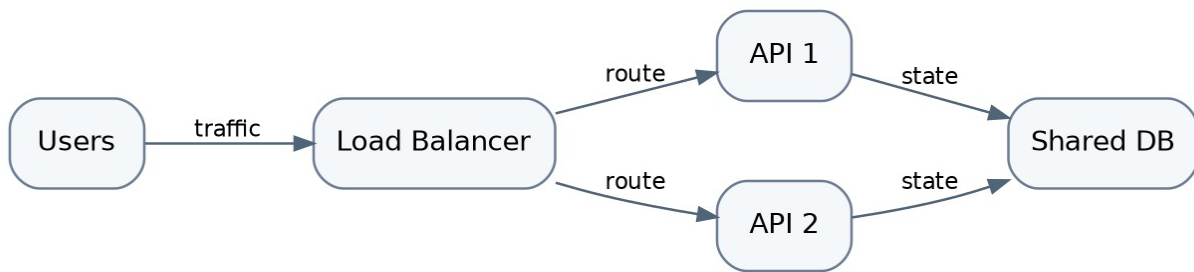


Figure 7.1 - Vertical vs Horizontal Scaling: simplified architecture

Three concepts to retain

- Vertical scaling keeps a single coherent machine but has hard hardware limits.
- Horizontal scaling requires statelessness or explicit state partitioning.
- Scaling is constrained by the least scalable stateful dependency.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Vertical vs Horizontal Scaling - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Scaling decision

A stateless API can scale to 20 replicas easily; a single primary database may remain the bottleneck.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If scaling becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Vertical vs Horizontal Scaling - Failure analysis

Failure scenario

You add ten API servers but throughput barely changes because all of them block on one saturated database.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Vertical vs Horizontal Scaling - Build, inspect, and read deeper

Small implementation / lab

Run the same stateless API on 1, 2, and 4 processes behind a load balancer; measure when the database becomes the bottleneck.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[NGINX load balancing](https://nginx.org/en/docs/http/load_balancing.html) - https://nginx.org/en/docs/http/load_balancing.html

[Google Cloud Architecture Framework](https://cloud.google.com/architecture/framework) - <https://cloud.google.com/architecture/framework>

Recall checklist

- Why does scaling exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 8 - Load Balancing and Health Checks

Why this exists. A load balancer is both a traffic distributor and a failure detector. Algorithm choice matters when requests differ in duration, servers differ in capacity, or sessions require affinity.

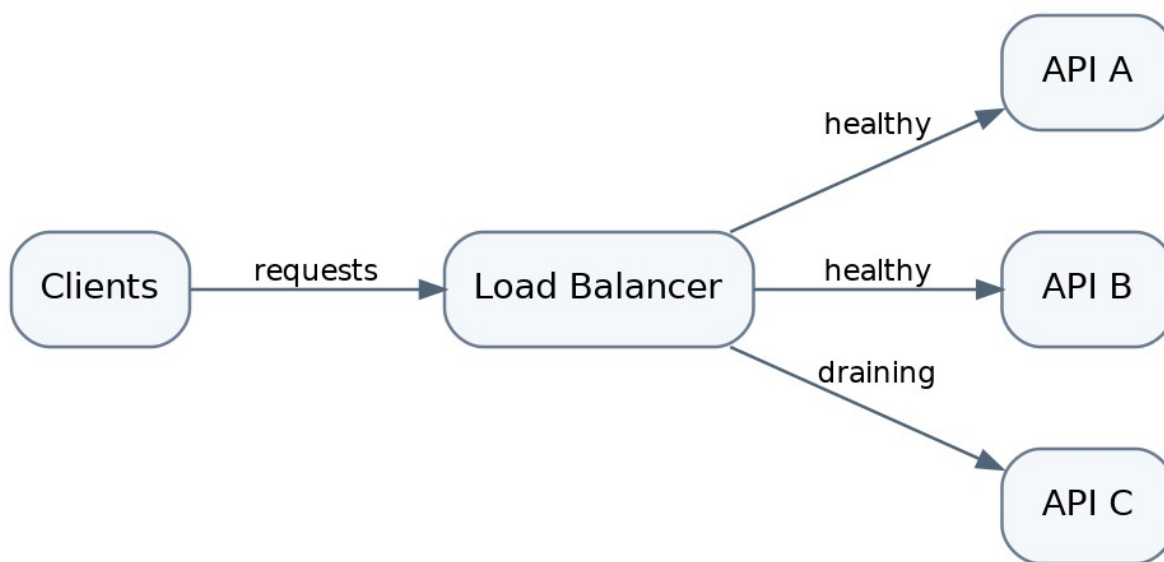


Figure 8.1 - Load Balancing and Health Checks: simplified architecture

Three concepts to retain

- Round robin assumes roughly equal servers and requests.
- Least-connections reacts better to variable request duration.
- Health checks and connection draining prevent traffic from reaching unhealthy or retiring nodes.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Load Balancing and Health Checks - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Weighted capacity

With weights 3:1:1, roughly 60% of traffic can be directed to the larger server.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If load balancing becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Load Balancing and Health Checks - Failure analysis

Failure scenario

A node remains in rotation after losing database connectivity because the health check only tests whether the process accepts TCP connections.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Load Balancing and Health Checks - Build, inspect, and read deeper

Small implementation / lab

Configure NGINX round robin, least_conn, and passive failure detection for three local backends with different artificial delays.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[NGINX load balancing](https://nginx.org/en/docs/http/load_balancing.html) - https://nginx.org/en/docs/http/load_balancing.html

Recall checklist

- Why does load balancing exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 9 - Stateless Services, Sessions, and Shared State

Why this exists. Horizontal scaling is easiest when any replica can serve any request. Sessions therefore belong either in signed client state or a shared datastore, not process memory.

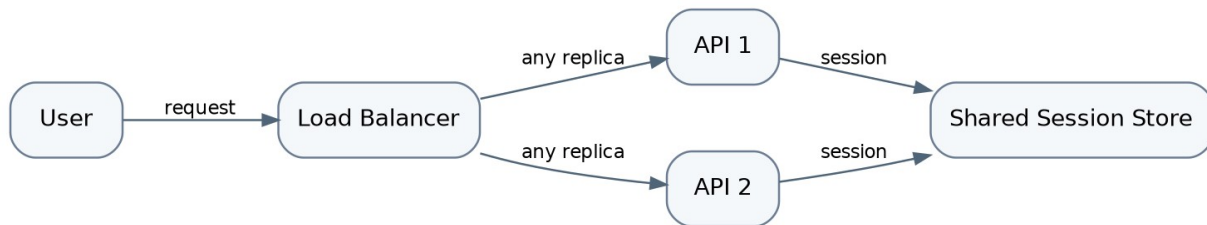


Figure 9.1 - Stateless Services, Sessions, and Shared State: simplified architecture

Three concepts to retain

- Process-local session state creates sticky-routing requirements.
- Shared session stores trade network calls for failover flexibility.
- Signed cookies reduce server state but expose size and revocation trade-offs.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Stateless Services, Sessions, and Shared State - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Failure

If user session exists only in API-3 memory and API-3 dies, the login session disappears.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If statelessness becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Stateless Services, Sessions, and Shared State - Failure analysis

Failure scenario

A rolling deploy randomly logs users out because sessions were stored in RAM on individual application servers.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Stateless Services, Sessions, and Shared State - Build, inspect, and read deeper

Small implementation / lab

Build the same login flow with in-memory sessions and Redis-backed sessions; kill one API process and compare behavior.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Redis documentation](https://redis.io/docs/latest/) - <https://redis.io/docs/latest/>

[OWASP Session Management](https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html) -

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

Recall checklist

- Why does statelessness exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 10 - Rate Limiting and Quotas

Why this exists. Rate limits protect shared systems from abuse, accidents, retry storms, and unfair tenants. A production limiter must define scope, algorithm, distributed state, and client feedback.



Figure 10.1 - Rate Limiting and Quotas: simplified architecture

Three concepts to retain

- Token bucket permits bursts while bounding long-term rate.
- Concurrency limits protect expensive long-duration operations that QPS limits miss.
- Rate-limit scope can be per user, API key, endpoint, tenant, region, or fleet.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Rate Limiting and Quotas - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Little's Law connection

$100 \text{ requests/s} * 5 \text{ s service time} = \text{about } 500 \text{ concurrent in-flight requests.}$

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If rate limiting becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Rate Limiting and Quotas - Failure analysis

Failure scenario

An endpoint accepts only 20 RPS but each request takes 60 seconds, exhausting all worker slots despite a seemingly low rate.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Rate Limiting and Quotas - Build, inspect, and read deeper

Small implementation / lab

Implement token-bucket and concurrency limiters; load-test them with short and long requests.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Stripe rate limiters](https://stripe.com/blog/rate-limiters) - <https://stripe.com/blog/rate-limiters>

[NGINX request limiting](https://nginx.org/en/docs/http/nginx_http_limit_req_module.html) - https://nginx.org/en/docs/http/nginx_http_limit_req_module.html

Recall checklist

- Why does rate limiting exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 11 - Autoscaling, Saturation, and Warm-Up Time

Why this exists. Autoscaling is a feedback-control problem. Reactive metrics, delayed capacity, workload heterogeneity, and startup time can create oscillation or persistent overload.

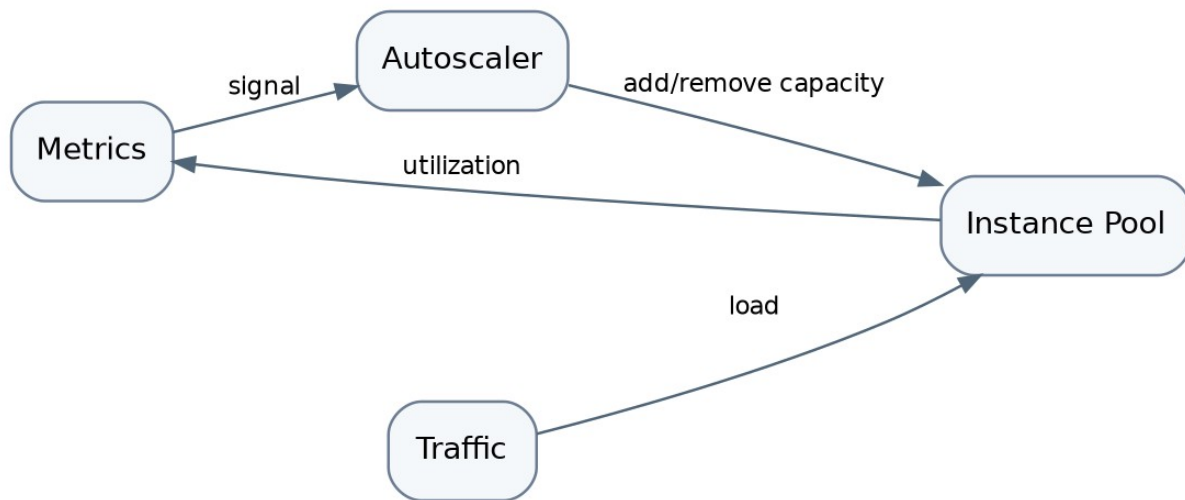


Figure 11.1 - Autoscaling, Saturation, and Warm-Up Time: simplified architecture

Three concepts to retain

- CPU is useful only if it tracks the constrained resource.
- Scale-out delay must be shorter than the time-to-overload or predictive scaling is needed.
- Scale-in requires draining and stabilization windows to avoid thrashing.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Autoscaling, Saturation, and Warm-Up Time - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Headroom

If instances take 4 minutes to warm and traffic can double in 1 minute, reactive autoscaling alone cannot prevent an overload window.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If autoscaling becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Autoscaling, Saturation, and Warm-Up Time - Failure analysis

Failure scenario

Autoscaler targets 70% average CPU while 5% of hosts are already saturated due to uneven request placement.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Autoscaling, Saturation, and Warm-Up Time - Build, inspect, and read deeper

Small implementation / lab

Create a toy autoscaler around queue depth; introduce a 30-second startup delay and observe overshoot/undershoot.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Google Cloud autoscaling concepts](https://cloud.google.com/compute/docs/autoscaler) - <https://cloud.google.com/compute/docs/autoscaler>

[Meta tail utilization](https://engineering.fb.com/2024/07/10/production-engineering/tail-utilization-ads-inference-meta/) - <https://engineering.fb.com/2024/07/10/production-engineering/tail-utilization-ads-inference-meta/>

Recall checklist

- Why does autoscaling exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 12 - Tail Latency and Fan-Out

Why this exists. At scale, a request often completes only when many subrequests complete. The maximum of many latency distributions amplifies tails even if each backend looks healthy alone.

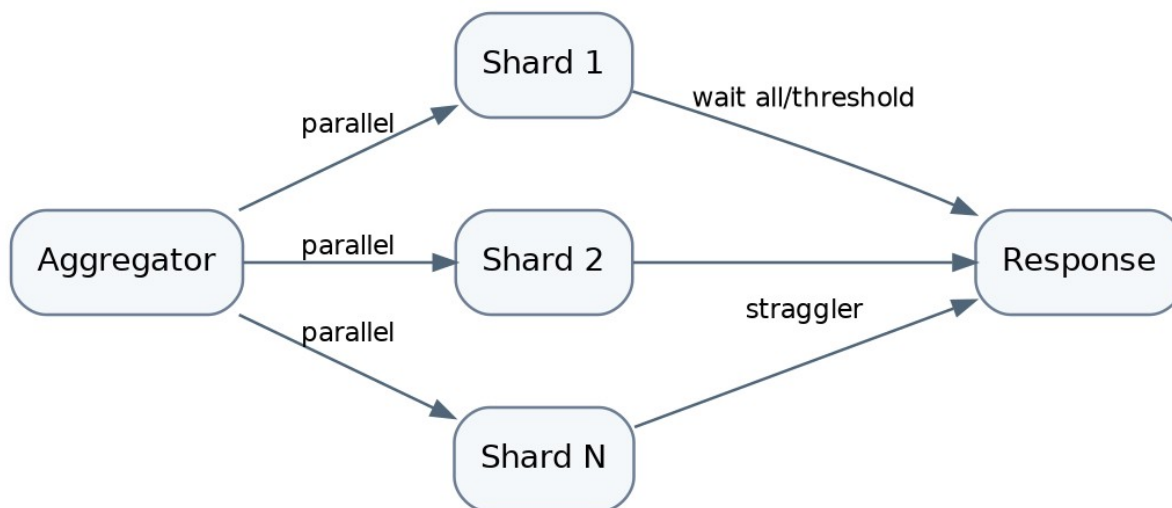


Figure 12.1 - Tail Latency and Fan-Out: simplified architecture

Three concepts to retain

- Fan-out converts modest per-shard tail probability into large end-to-end tail probability.
- Hedged requests trade extra load for lower tail latency.
- Correlated slowness cannot be fixed by naive duplication and may worsen overload.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Tail Latency and Fan-Out - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Tail math

If each of 100 independent shards responds within 100 ms with probability 0.99, all 100 do so with probability $0.99^{100} \approx 36.6\%$.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If tail latency becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Tail Latency and Fan-Out - Failure analysis

Failure scenario

Search fans out to 100 shards; one slow shard dominates p99 even though every shard individually meets its local p99.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Tail Latency and Fan-Out - Build, inspect, and read deeper

Small implementation / lab

Simulate 50 shard latencies with a log-normal distribution; compare end-to-end max latency with and without a delayed hedge.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Google Research - The Tail at Scale](https://research.google/pubs/the-tail-at-scale/) - <https://research.google/pubs/the-tail-at-scale/>

Recall checklist

- Why does tail latency exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

PART III - DATA SYSTEMS

The goal of this part is to build a reusable mental model, not memorize a product stack. Every mechanism is tied to an invariant, a bottleneck, or a failure mode.

Chapter 13 - Relational Data Modeling and Constraints

Why this exists. Relational design encodes business invariants close to the data. Keys, foreign keys, unique constraints, and normalization prevent many classes of application-level corruption.

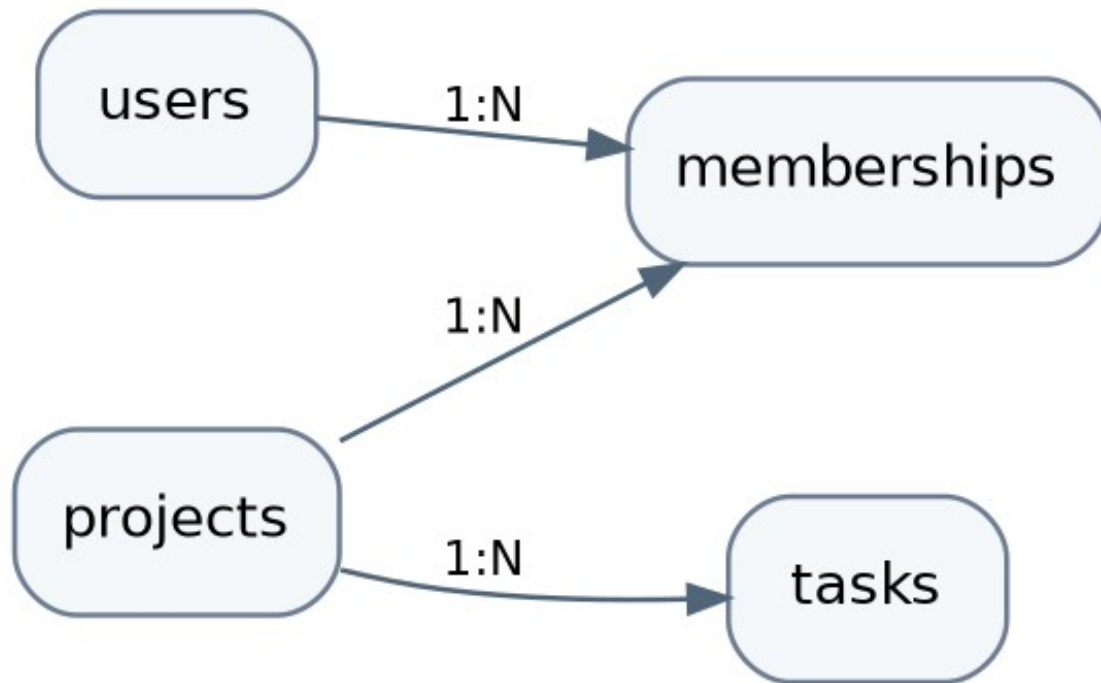


Figure 13.1 - Relational Data Modeling and Constraints: simplified architecture

Three concepts to retain

- Primary keys establish identity; foreign keys encode relationships.
- Unique and check constraints make invariants executable.
- Normalization reduces update anomalies; denormalization trades duplication for read speed.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Relational Data Modeling and Constraints - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Invariant

UNIQUE(email) is stronger than “the application checks for duplicates before insert” under concurrency.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If relational databases becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Relational Data Modeling and Constraints - Failure analysis

Failure scenario

Two API replicas simultaneously create the same username because uniqueness exists only in application code.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Relational Data Modeling and Constraints - Build, inspect, and read deeper

Small implementation / lab

Model users, projects, memberships, and tasks in PostgreSQL with PK/FK/UNIQUE/CHECK constraints; intentionally violate each one.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[PostgreSQL constraints](https://www.postgresql.org/docs/current/ddl-constraints.html) - <https://www.postgresql.org/docs/current/ddl-constraints.html>

Recall checklist

- Why does relational databases exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 14 - Transactions and ACID

Why this exists. Transactions group related state transitions so partial execution does not leak into the database. The purpose is not ceremony; it is to preserve invariants across crashes and concurrent requests.

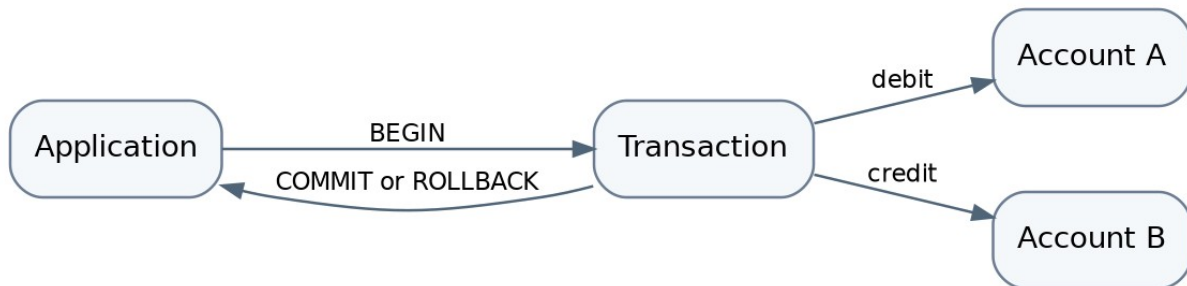


Figure 14.1 - Transactions and ACID: simplified architecture

Three concepts to retain

- Atomicity gives all-or-nothing behavior.
- Isolation controls interaction among concurrent transactions.
- Durability means committed state survives expected failures.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Transactions and ACID - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Bank transfer

Debit A and credit B must commit together; a crash between two autocommit statements can destroy money.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If transactions becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Transactions and ACID - Failure analysis

Failure scenario

An order row commits, inventory decrement fails, and the user receives a confirmed order for stock that was never reserved.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Transactions and ACID - Build, inspect, and read deeper

Small implementation / lab

Implement a money transfer with BEGIN/COMMIT/ROLLBACK; inject an exception after debit and verify the invariant.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[PostgreSQL transaction tutorial](https://www.postgresql.org/docs/current/tutorial-transactions.html) - <https://www.postgresql.org/docs/current/tutorial-transactions.html>

Recall checklist

- Why does transactions exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 15 - Isolation, MVCC, and Concurrency Anomalies

Why this exists. Concurrent transactions can each be locally correct while the combined result violates expectations. Isolation levels choose which anomalies are allowed in exchange for concurrency.

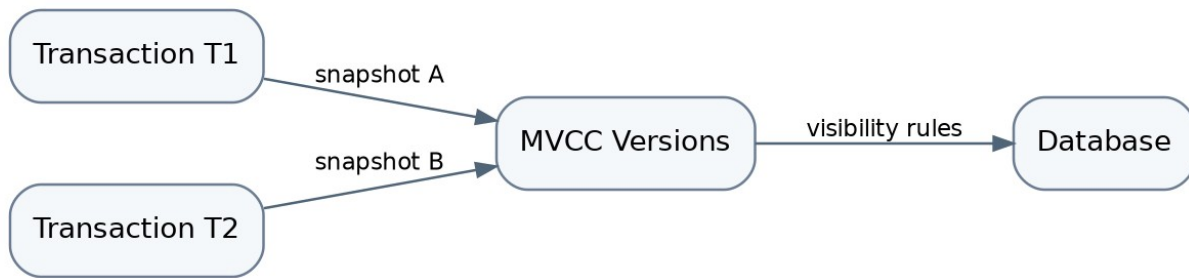


Figure 15.1 - Isolation, MVCC, and Concurrency Anomalies: simplified architecture

Three concepts to retain

- Read Committed sees a fresh committed snapshot per statement.
- Repeatable Read stabilizes a transaction snapshot but concurrent writes can still require care.
- Serializable aims for behavior equivalent to some serial execution and may abort transactions.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Isolation, MVCC, and Concurrency Anomalies - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Write skew

Two doctors each observe the other on call, then both go off call. Each transaction saw a valid snapshot; the final invariant is broken.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If transaction isolation becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Isolation, MVCC, and Concurrency Anomalies - Failure analysis

Failure scenario

A “check then update” workflow passes in two concurrent transactions and oversells a limited resource.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Isolation, MVCC, and Concurrency Anomalies - Build, inspect, and read deeper

Small implementation / lab

Reproduce non-repeatable reads and serialization failures using two psql sessions and different isolation levels.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[PostgreSQL transaction isolation](https://www.postgresql.org/docs/current/transaction-iso.html) - <https://www.postgresql.org/docs/current/transaction-iso.html>

[PostgreSQL SET TRANSACTION](https://www.postgresql.org/docs/current/sql-set-transaction.html) - <https://www.postgresql.org/docs/current/sql-set-transaction.html>

Recall checklist

- Why does transaction isolation exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 16 - Indexes: B-Tree, Hash, GIN, BRIN

Why this exists. An index trades extra storage and write cost for faster access paths. The correct index follows the query shape, not the column that “looks important.”

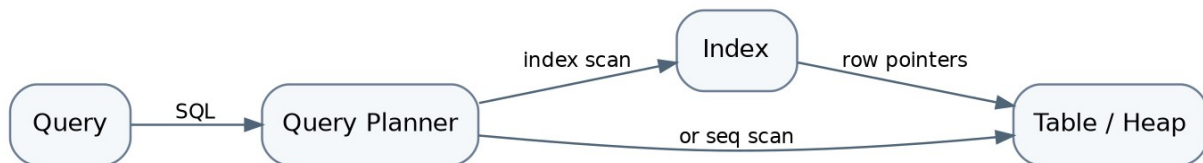


Figure 16.1 - Indexes: B-Tree, Hash, GIN, BRIN: simplified architecture

Three concepts to retain

- B-tree supports equality and ordered range queries.
- GIN is effective for composite values such as arrays/full text.
- BRIN summarizes physical ranges and can be tiny for naturally ordered large tables.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Indexes: B-Tree, Hash, GIN, BRIN - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Complexity intuition

An indexed lookup may avoid scanning millions of rows, but every additional index increases write amplification.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If database indexes becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Indexes: B-Tree, Hash, GIN, BRIN - Failure analysis

Failure scenario

A table has six indexes “just in case”; write throughput collapses and most indexes are never used.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Indexes: B-Tree, Hash, GIN, BRIN - Build, inspect, and read deeper

Small implementation / lab

Create a 1M-row Postgres table; compare EXPLAIN ANALYZE before/after indexes on equality, range, and text-search queries.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[PostgreSQL index types](https://www.postgresql.org/docs/current/indexes-types.html) - <https://www.postgresql.org/docs/current/indexes-types.html>

[PostgreSQL EXPLAIN](https://www.postgresql.org/docs/current/using-explain.html) - <https://www.postgresql.org/docs/current/using-explain.html>

Recall checklist

- Why does database indexes exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 17 - Replication, Failover, and Replication Lag

Why this exists. Replication increases read capacity and availability, but asynchronous copies create lag and failover ambiguity. A replica is not automatically an up-to-date replacement for a failed primary.

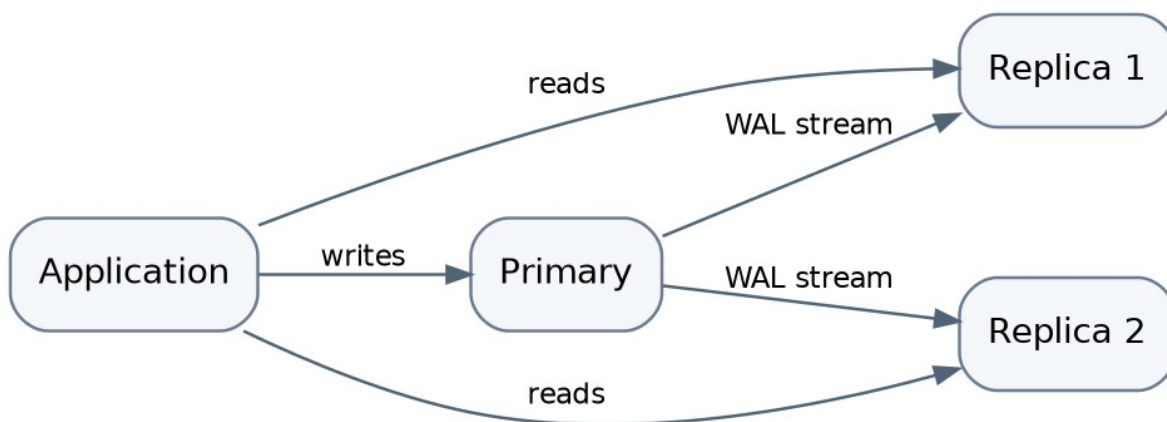


Figure 17.1 - Replication, Failover, and Replication Lag: simplified architecture

Three concepts to retain

- Streaming replication copies a write log from primary to replicas.
- Read replicas can serve stale data depending on lag.
- Failover requires leader selection, fencing, and recovery of missing writes.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Replication, Failover, and Replication Lag - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Read-after-write

User updates profile on primary, then GET is routed to a lagging replica and sees old data.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If replication becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Replication, Failover, and Replication Lag - Failure analysis

Failure scenario

Primary fails after acknowledging a write that has not reached any candidate replica; automatic promotion loses acknowledged data.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Replication, Failover, and Replication Lag - Build, inspect, and read deeper

Small implementation / lab

Run PostgreSQL primary/replica locally or inspect replication metrics; create artificial delay and measure stale reads.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[PostgreSQL high availability](https://www.postgresql.org/docs/current/high-availability.html) - <https://www.postgresql.org/docs/current/high-availability.html>

Recall checklist

- Why does replication exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 18 - Caching: Cache-Aside, TTL, and Stampedes

Why this exists. Caching is a load-shaping system with correctness consequences. The real problems are invalidation, stale data, stampedes, hot keys, and behavior when the cache disappears.

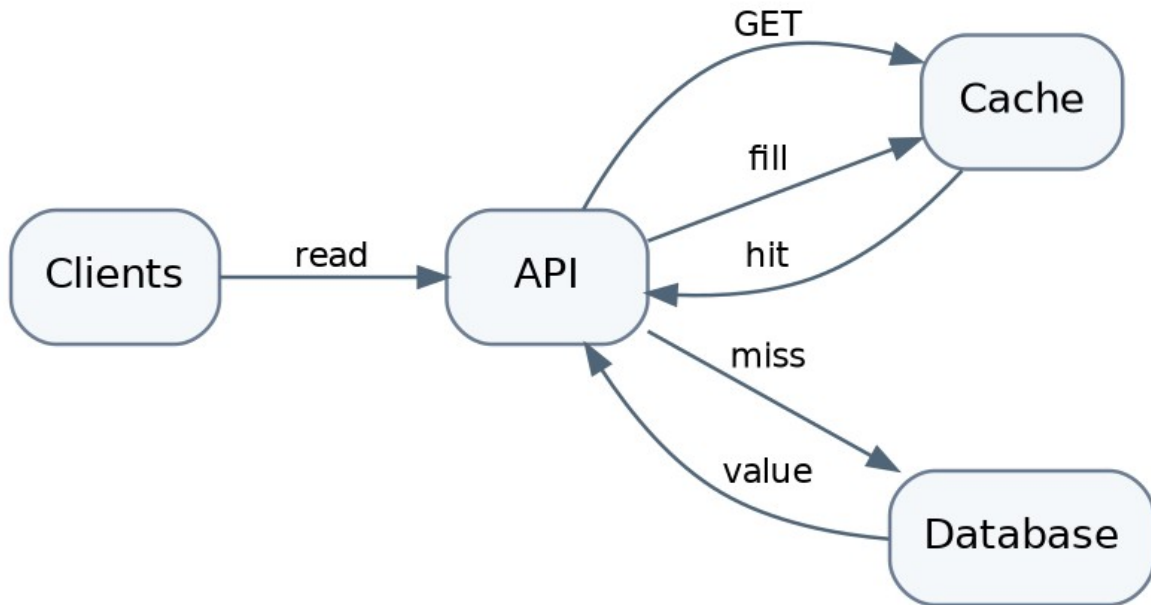


Figure 18.1 - Caching: Cache-Aside, TTL, and Stampedes: simplified architecture

Three concepts to retain

- Cache-aside lets applications control fill and invalidation.
- TTL bounds staleness but can synchronize expirations and create stampedes.
- Request coalescing and jitter reduce duplicate recomputation.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Caching: Cache-Aside, TTL, and Stampedes - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Load reduction

At 90% hit ratio, a database that would receive 100k reads/s sees about 10k reads/s.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If caching becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Caching: Cache-Aside, TTL, and Stampedes - Failure analysis

Failure scenario

A popular key expires at noon; 20k requests simultaneously miss and overwhelm the database.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Caching: Cache-Aside, TTL, and Stampedes - Build, inspect, and read deeper

Small implementation / lab

Implement cache-aside with Redis; compare fixed TTL to jittered TTL and add a single-flight/request-coalescing lock.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Redis eviction policies](https://redis.io/docs/latest/develop/reference/eviction/) - <https://redis.io/docs/latest/develop/reference/eviction/>

[Meta Scaling Memcache](https://engineering.fb.com/2013/04/15/core-infra/scaling-memcache-at-facebook/) - <https://engineering.fb.com/2013/04/15/core-infra/scaling-memcache-at-facebook/>

Recall checklist

- Why does caching exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 19 - Redis: Persistence, Eviction, and Failure

Why this exists. Redis can be a cache, ephemeral store, coordination primitive, or durable-ish data system depending on configuration. The system design must state which role it plays.

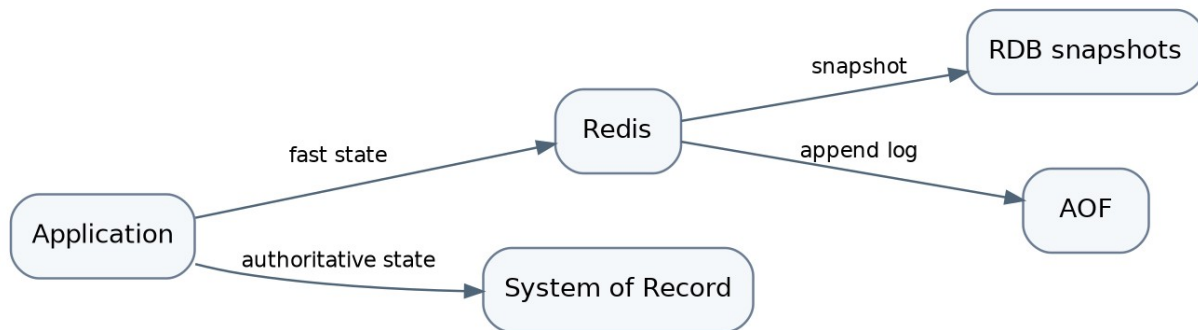


Figure 19.1 - Redis: Persistence, Eviction, and Failure: simplified architecture

Three concepts to retain

- RDB snapshots and AOF have different durability/performance characteristics.
- Eviction policies define behavior under memory pressure.
- Using Redis as the only source of truth requires a very different recovery plan than using it as a cache.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Redis: Persistence, Eviction, and Failure - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Failure contract

If Redis is only a cache, losing it should hurt latency, not correctness.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If Redis becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Redis: Persistence, Eviction, and Failure - Failure analysis

Failure scenario

A “cache” accidentally becomes the only store for session-critical data; a restart turns into a correctness incident.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Redis: Persistence, Eviction, and Failure - Build, inspect, and read deeper

Small implementation / lab

Configure Redis with no persistence, RDB, and AOF; restart the server and document what survives.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Redis persistence](https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/) - https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

[Redis eviction](https://redis.io/docs/latest/develop/reference/eviction/) - <https://redis.io/docs/latest/develop/reference/eviction/>

Recall checklist

- Why does Redis exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 20 - Sharding and Partition-Key Design

Why this exists. Sharding distributes state across machines, but the partition key determines locality, hotspot risk, transaction boundaries, and future migration pain.

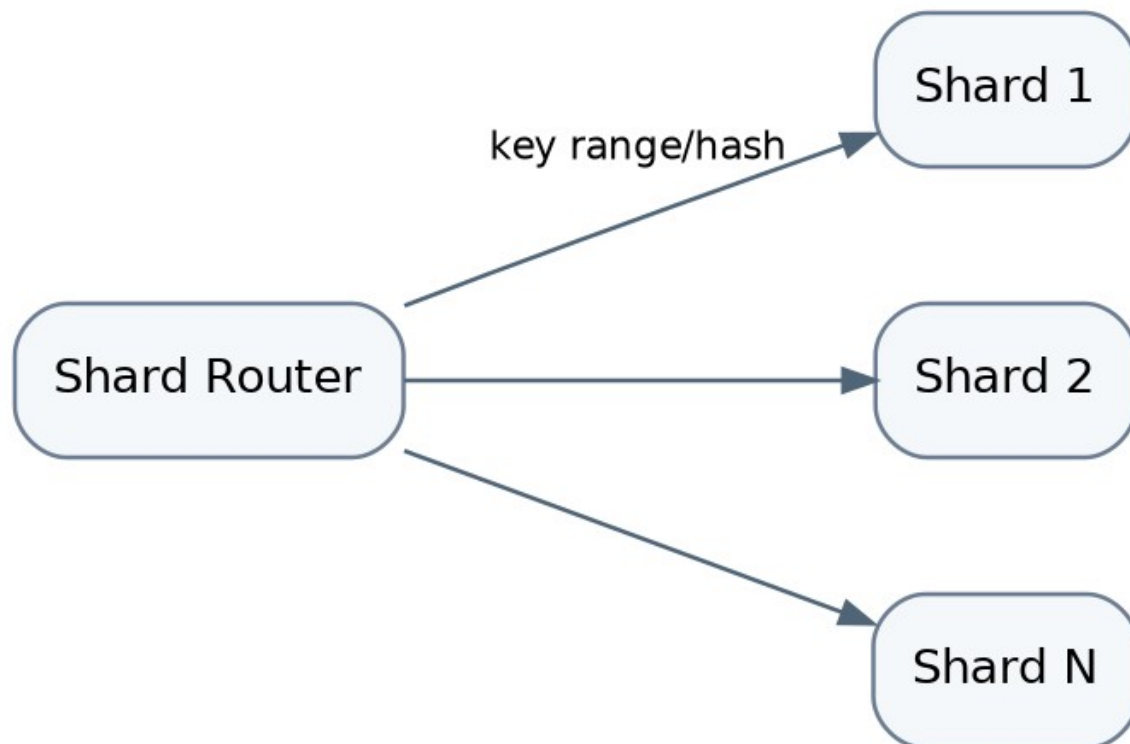


Figure 20.1 - Sharding and Partition-Key Design: simplified architecture

Three concepts to retain

- Hash sharding balances random keys but destroys range locality.
- Range sharding preserves ordered access but risks hotspots.
- Tenant/entity-based sharding can align data ownership with product boundaries.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Sharding and Partition-Key Design - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Skew

Even with equal key counts, one celebrity tenant can generate most of the traffic to a single shard.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If sharding becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Sharding and Partition-Key Design - Failure analysis

Failure scenario

Shard by `user_id`, then discover that organization-wide queries and transactions fan out across dozens of shards.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Sharding and Partition-Key Design - Build, inspect, and read deeper

Small implementation / lab

Partition a synthetic workload by hash, range, and tenant; compare key balance, request balance, and fan-out cost.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[GitHub database partitioning](https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/) - <https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/>

Recall checklist

- Why does sharding exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 21 - Consistent Hashing and Membership Churn

Why this exists. Consistent hashing minimizes remapping when nodes join or leave, preserving cache/state locality in elastic fleets. It does not automatically solve hot keys or unequal node capacity.

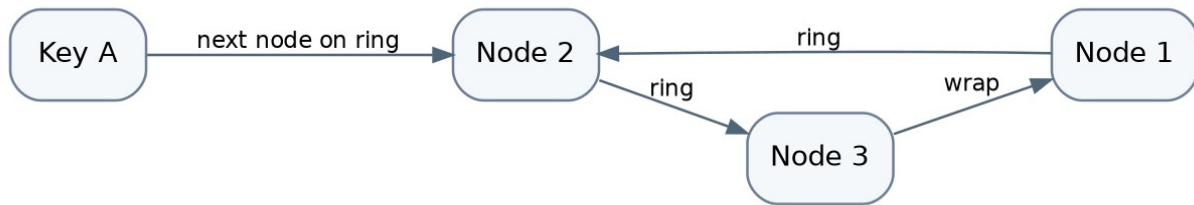


Figure 21.1 - Consistent Hashing and Membership Churn: simplified architecture

Three concepts to retain

- Keys and nodes share a hash ring.
- Virtual nodes improve distribution and allow weighted capacity.
- Membership changes move only nearby ownership ranges rather than rehashing everything.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Consistent Hashing and Membership Churn - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Remapping

With naive $\text{hash}(\text{key}) \% N$, changing N can remap most keys. A consistent ring changes only a fraction near the changed node.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If consistent hashing becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Consistent Hashing and Membership Churn - Failure analysis

Failure scenario

Autoscaling a cache from 100 to 110 nodes causes a near-total cache miss storm because modulo ownership changes globally.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Consistent Hashing and Membership Churn - Build, inspect, and read deeper

Small implementation / lab

Implement modulo hashing and a consistent-hash ring with 100 virtual nodes/server; compare remap rate after adding/removing one node.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Cloudflare - shard and conquer](https://blog.cloudflare.com/eliminating-cold-starts-2-shard-and-conquer/) - <https://blog.cloudflare.com/eliminating-cold-starts-2-shard-and-conquer/>

Recall checklist

- Why does consistent hashing exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 22 - NoSQL Data Models: KV, Document, Column, Graph

Why this exists. NoSQL is not one model. The right datastore follows access patterns, consistency needs, data shape, and operational constraints.

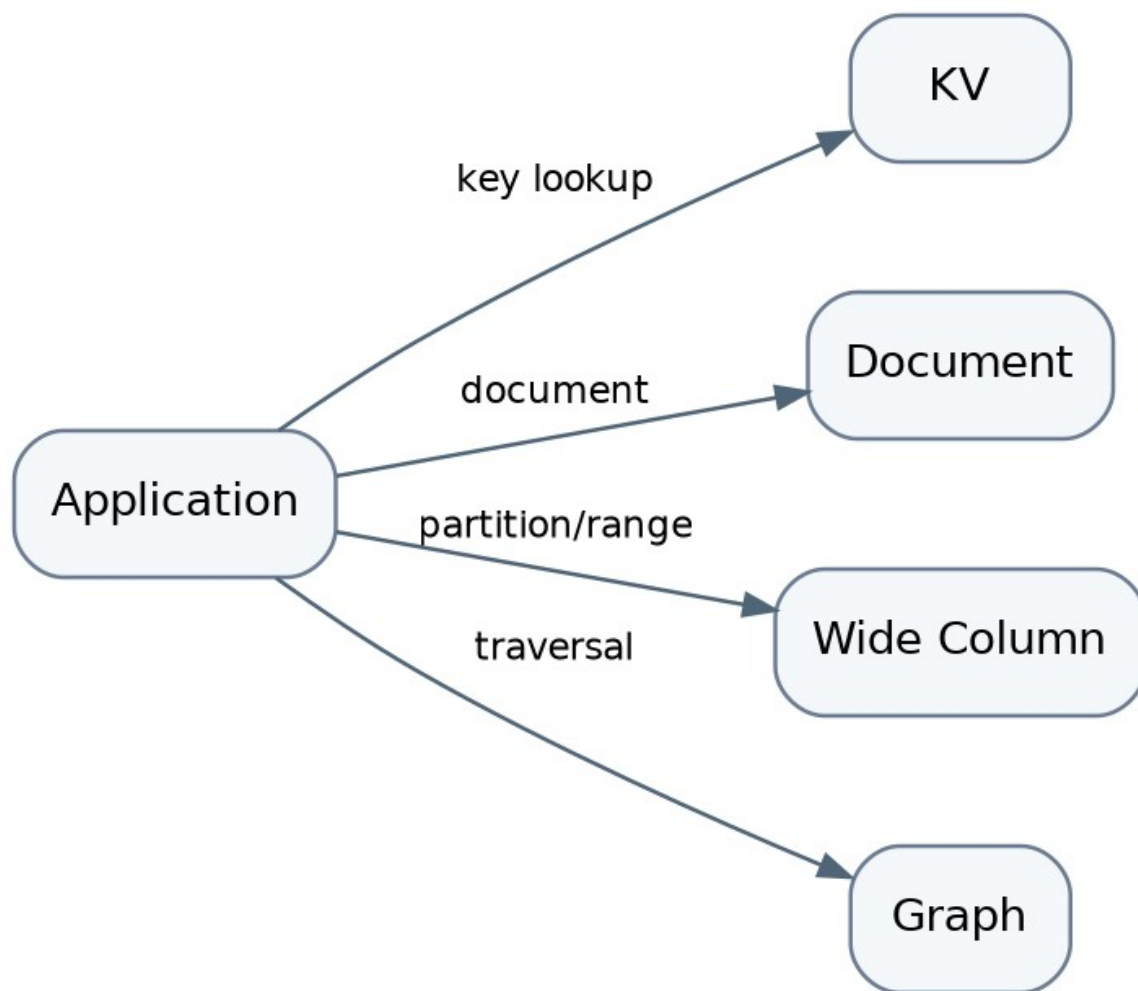


Figure 22.1 - NoSQL Data Models: KV, Document, Column, Graph: simplified architecture

Three concepts to retain

- Key-value stores optimize direct lookup by key.
- Document stores preserve flexible nested records.
- Wide-column and graph stores target sparse large-scale rows and relationship traversal respectively.

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

NoSQL Data Models: KV, Document, Column, Graph - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Choice

If every request is `Get(user_id)`, joins may be unnecessary overhead; if cross-entity transactional invariants dominate, a relational DB may remain best.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If NoSQL becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

NoSQL Data Models: KV, Document, Column, Graph - Failure analysis

Failure scenario

A team chooses a document database for “scale” but later requires multi-entity constraints and complex joins across documents.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

NoSQL Data Models: KV, Document, Column, Graph - Build, inspect, and read deeper

Small implementation / lab

Model the same social feature in relational, document, KV, and graph forms and list which queries become simpler/harder.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Dynamo paper](https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf) - <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

[Bigtable paper](https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/) - <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>

Recall checklist

- Why does NoSQL exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 23 - Object Storage, Content Addressing, and Large Blobs

Why this exists. Large immutable blobs have different economics and access patterns from transactional rows. Object storage separates object identity, metadata, and durable blob placement.

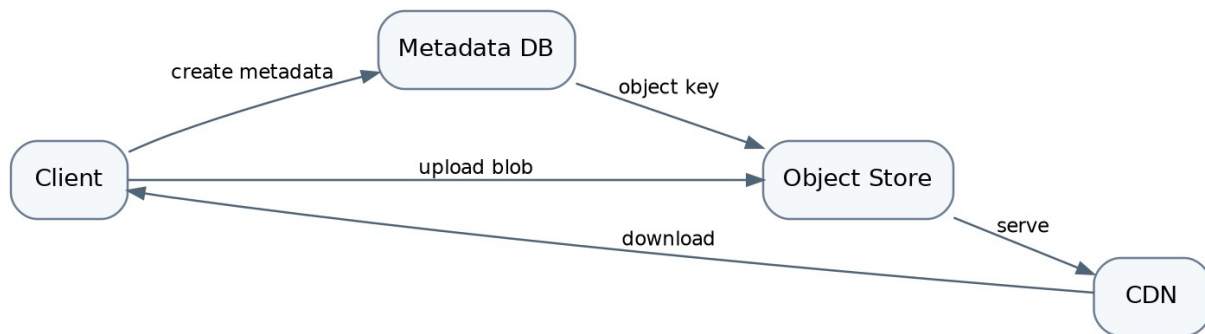


Figure 23.1 - Object Storage, Content Addressing, and Large Blobs: simplified architecture

Three concepts to retain

- Object stores are optimized for large durable objects, not low-latency row updates.
- Content hashes enable deduplication and integrity checks.
- Metadata usually belongs in a separate indexed database.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Object Storage, Content Addressing, and Large Blobs - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Bandwidth

A 4 GB video uploaded twice is 8 GB of ingress unless deduplication/chunking prevents duplication.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If object storage becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Object Storage, Content Addressing, and Large Blobs - Failure analysis

Failure scenario

A team stores multi-megabyte videos directly in relational rows; backups, replication, and query I/O become painfully expensive.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Object Storage, Content Addressing, and Large Blobs - Build, inspect, and read deeper

Small implementation / lab

Build a mini blob store: content hash -> file/object path, metadata in Postgres, and deduplicate repeated uploads.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Amazon S3 concepts](https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html) - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

[Meta Haystack](https://engineering.fb.com/2009/04/30/core-infra/needle-in-a-haystack-efficient-storage-of-billions-of-photos/) - <https://engineering.fb.com/2009/04/30/core-infra/needle-in-a-haystack-efficient-storage-of-billions-of-photos/>

Recall checklist

- Why does object storage exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

PART IV - DISTRIBUTED AND EVENT-DRIVEN SYSTEMS

The goal of this part is to build a reusable mental model, not memorize a product stack. Every mechanism is tied to an invariant, a bottleneck, or a failure mode.

Chapter 24 - Logical Time, Causality, and Lamport Clocks

Why this exists. Distributed systems cannot rely on a perfectly synchronized global wall clock. Logical clocks model happened-before relationships without pretending every event has one absolute global order.

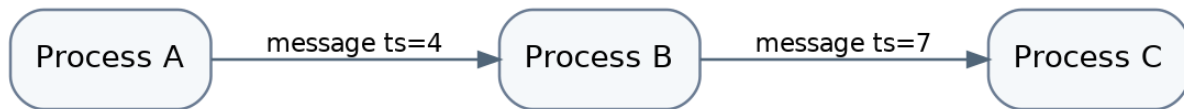


Figure 24.1 - Logical Time, Causality, and Lamport Clocks: simplified architecture

Three concepts to retain

- Lamport clocks preserve causal order: if a happened-before b , then $C(a) < C(b)$.
- The reverse does not prove causality.
- Vector clocks can represent concurrent histories more precisely at higher metadata cost.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Logical Time, Causality, and Lamport Clocks - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Lamport update

On local event increment; on receive set $\text{clock} = \max(\text{local}, \text{received}) + 1$.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If logical clocks becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Logical Time, Causality, and Lamport Clocks - Failure analysis

Failure scenario

Two regions order concurrent updates by wall-clock timestamp; clock skew causes an older write to overwrite a newer causal update.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Logical Time, Causality, and Lamport Clocks - Build, inspect, and read deeper

Small implementation / lab

Simulate three processes exchanging messages and implement Lamport timestamps; identify which events are causally related vs concurrent.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Lamport paper](https://lamport.azurewebsites.net/pubs/time-clocks.pdf) - <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>

Recall checklist

- Why does logical clocks exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 25 - Quorums and Replicated Reads/Writes

Why this exists. Quorum systems trade consistency, availability, and latency by choosing how many replicas participate in each read and write.

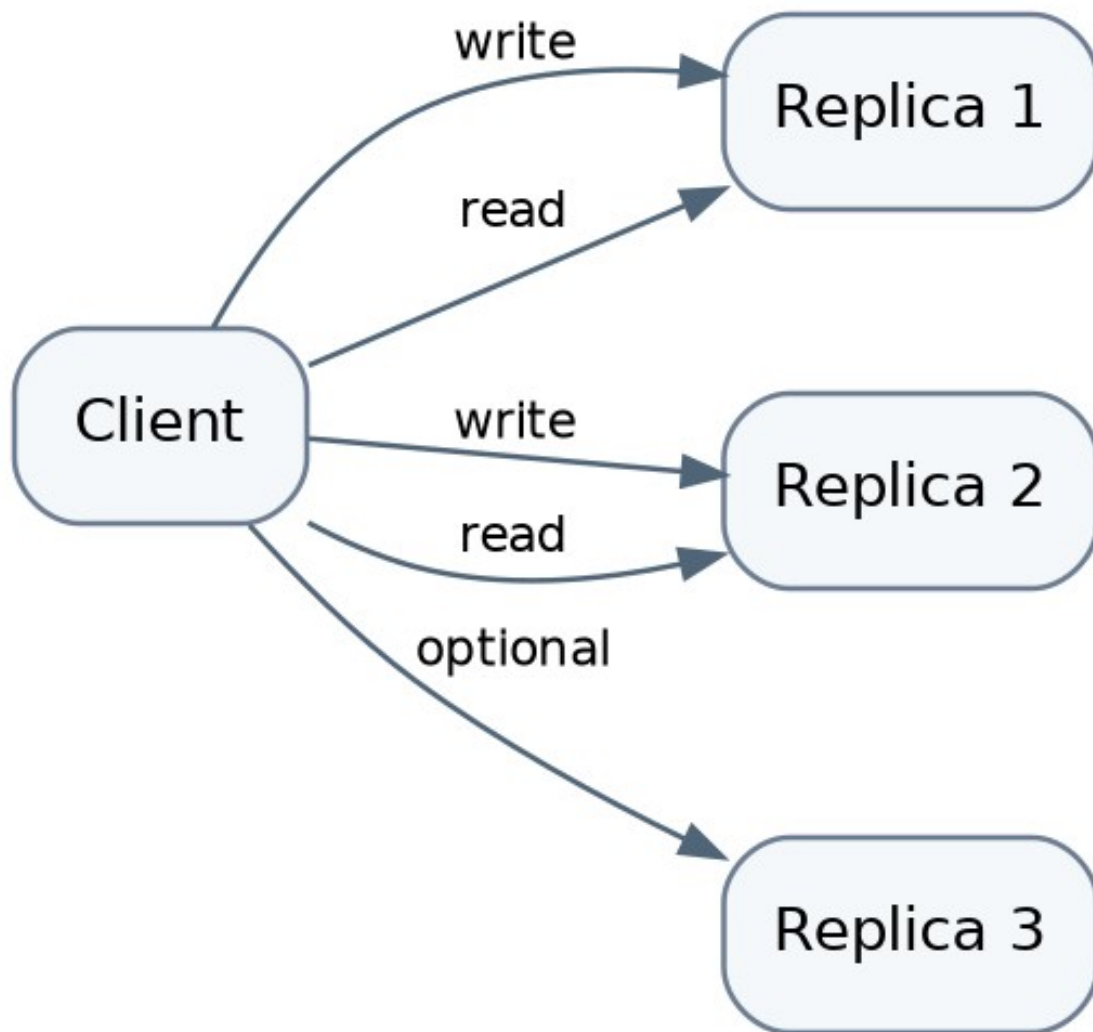


Figure 25.1 - Quorums and Replicated Reads/Writes: simplified architecture

Three concepts to retain

- With N replicas, common reasoning uses R and W quorum sizes.
- $R + W > N$ creates overlap between read and write quorums.

- Node failures, sloppy quorums, hinted handoff, and conflict resolution complicate the simple formula.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Quorums and Replicated Reads/Writes - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Example

$N=3$, $W=2$, $R=2$ guarantees a read quorum overlaps the latest successful write quorum under the simplified model.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If quorum replication becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Quorums and Replicated Reads/Writes - Failure analysis

Failure scenario

A system uses $W=1$ and $R=1$ for low latency, then users immediately read stale state from a different replica.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Quorums and Replicated Reads/Writes - Build, inspect, and read deeper

Small implementation / lab

Simulate N=5 replicas with variable latency and compare response time/availability for several R/W combinations.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Dynamo paper](https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf) - <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

Recall checklist

- Why does quorum replication exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 26 - Consensus and Raft

Why this exists. Consensus lets machines maintain one coherent replicated log despite crashes and message delays. Raft separates the problem into leader election, log replication, and safety.

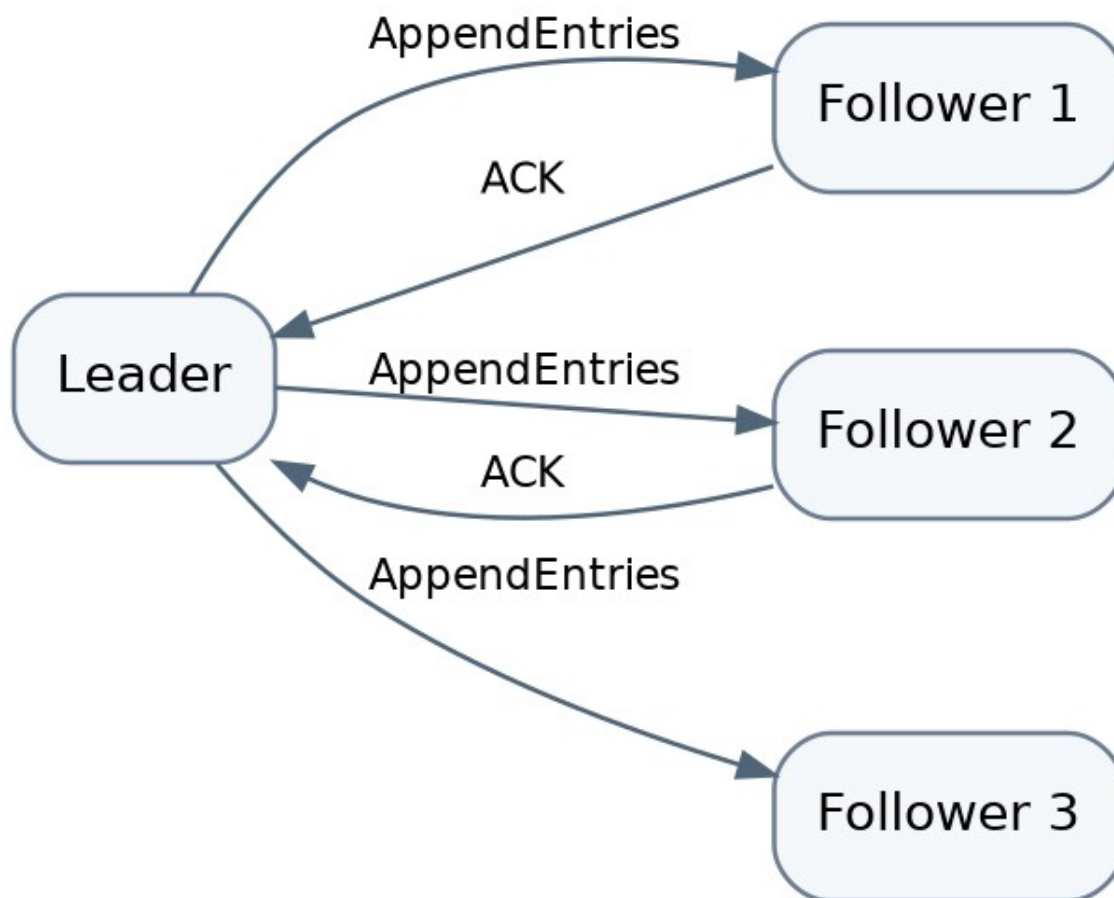


Figure 26.1 - Consensus and Raft: simplified architecture

Three concepts to retain

- Terms and randomized elections choose leaders.
- The leader appends entries and commits after majority replication.
- Safety rules prevent an elected leader from losing committed entries.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Consensus and Raft - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Majority

In a five-node cluster, majority is three; two failures can be tolerated for progress.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If consensus becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Consensus and Raft - Failure analysis

Failure scenario

Network partition creates a minority old leader and majority new leader; without term/majority safety both sides could accept conflicting authoritative writes.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Consensus and Raft - Build, inspect, and read deeper

Small implementation / lab

Use an existing Raft visualization or implement a toy election/log simulator; kill the leader and observe election and log catch-up.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Raft paper](https://raft.github.io/raft.pdf) - <https://raft.github.io/raft.pdf>

[Raft site](https://raft.github.io/) - <https://raft.github.io/>

Recall checklist

- Why does consensus exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 27 - Distributed IDs and Snowflake

Why this exists. Distributed IDs must remain unique without a central bottleneck. Time-based schemes can also preserve rough ordering while encoding worker identity and a per-time-unit sequence.

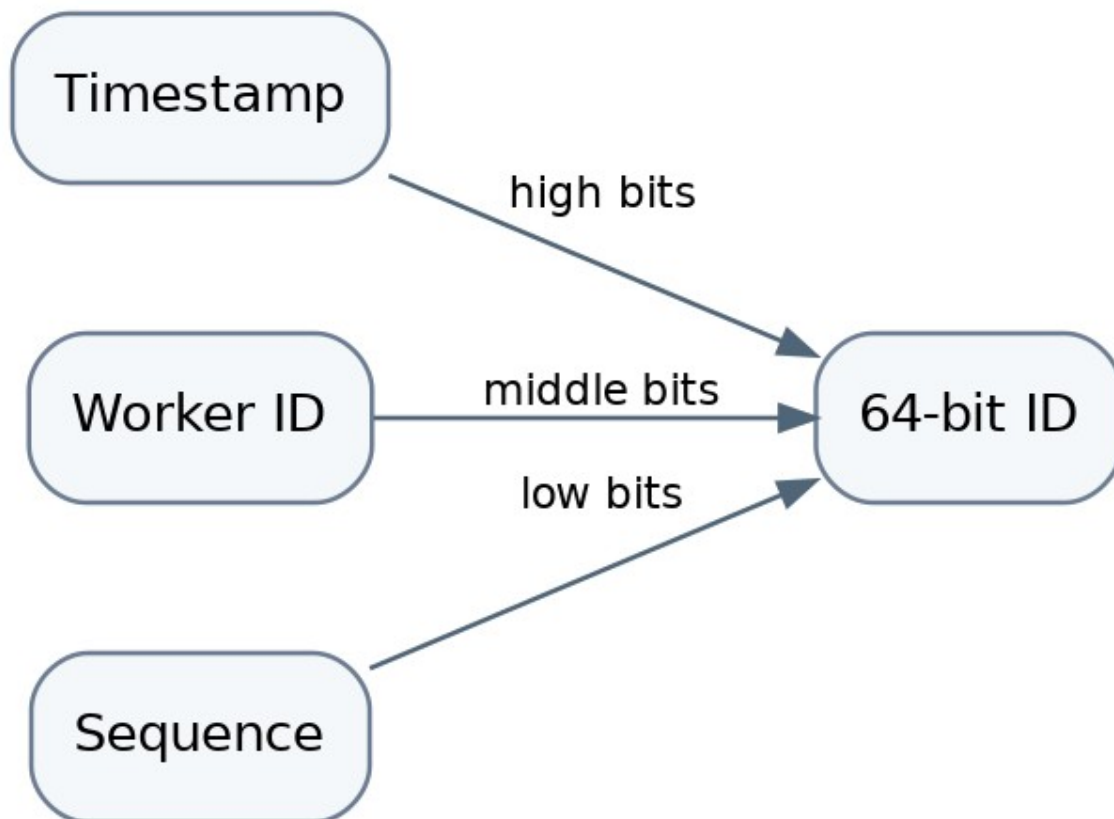


Figure 27.1 - Distributed IDs and Snowflake: simplified architecture

Three concepts to retain

- A Snowflake-style ID combines timestamp, worker, and sequence bits.
- Clock rollback and worker-ID allocation are correctness concerns.
- Random UUIDs avoid coordination but have different locality/index behavior.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Distributed IDs and Snowflake - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Bit budget

41 timestamp bits, 10 worker bits, and 12 sequence bits imply 1024 workers and 4096 IDs per millisecond per worker.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If distributed IDs becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Distributed IDs and Snowflake - Failure analysis

Failure scenario

Two machines accidentally receive the same worker ID and generate duplicate IDs during the same millisecond.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Distributed IDs and Snowflake - Build, inspect, and read deeper

Small implementation / lab

Implement a 64-bit Snowflake generator, force >4096 IDs/ms and a simulated clock rollback, and define the policy.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Twitter Snowflake repository](https://github.com/twitter-archive/snowflake) - <https://github.com/twitter-archive/snowflake>

Recall checklist

- Why does distributed IDs exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 28 - Queues, Backpressure, and Queueing Theory

Why this exists. Queues decouple arrival from processing and absorb short bursts, but they cannot create capacity. If average arrival exceeds average service, backlog grows without bound.

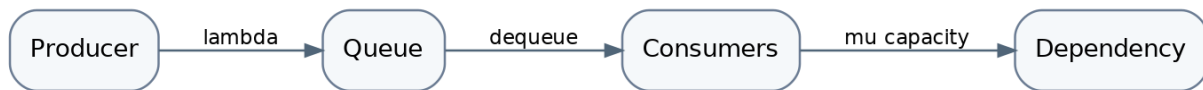


Figure 28.1 - Queues, Backpressure, and Queueing Theory: simplified architecture

Three concepts to retain

- Backlog growth is approximately $\lambda - \mu$ when arrival exceeds service.
- Queue depth alone is less useful than age-of-oldest-message for user impact.
- Backpressure should slow producers or shed work before storage becomes an unbounded waiting room.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Queues, Backpressure, and Queueing Theory - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Little's Law

$L = \lambda W$ links average items in system, arrival rate, and average time in system.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If queues becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Queues, Backpressure, and Queueing Theory - Failure analysis

Failure scenario

Workers process 60 jobs/s while producers enqueue 100 jobs/s; after one hour, about 144,000 additional jobs are waiting.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Queues, Backpressure, and Queueing Theory - Build, inspect, and read deeper

Small implementation / lab

Build a producer/consumer queue simulator; vary arrival rate, worker count, and service time; graph backlog and age.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[AWS - Avoiding insurmountable queue backlogs](https://aws.amazon.com/builders-library/avoiding-insurmountable-queue-backlogs/) - <https://aws.amazon.com/builders-library/avoiding-insurmountable-queue-backlogs/>

Recall checklist

- Why do queues exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 29 - Kafka: Logs, Partitions, and Consumer Groups

Why this exists. Kafka is a distributed append-only log. Partitioning provides scale and ordering boundaries; consumer groups divide partitions among workers and offsets represent processing position.

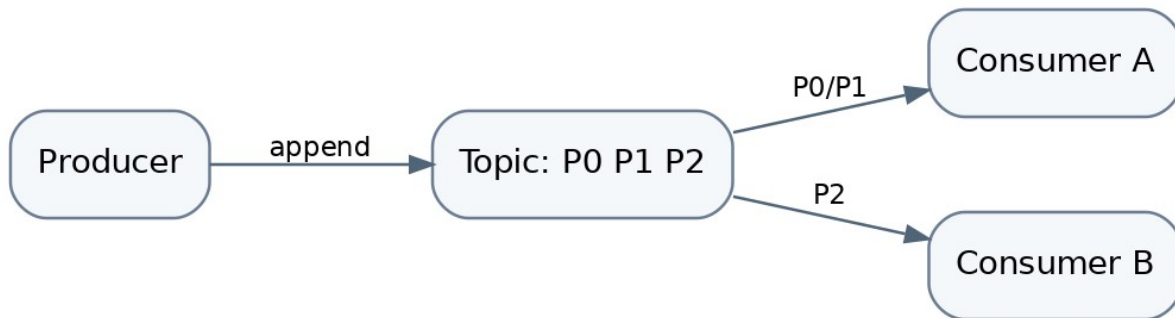


Figure 29.1 - Kafka: Logs, Partitions, and Consumer Groups: simplified architecture

Three concepts to retain

- Ordering is guaranteed within a partition, not across a topic.
- A consumer group assigns each partition to at most one active consumer at a time.
- Offsets make replay and recovery explicit.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Kafka: Logs, Partitions, and Consumer Groups - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Parallelism ceiling

A topic with 12 partitions can actively feed at most 12 consumers in one group without idle members.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If Kafka becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Kafka: Logs, Partitions, and Consumer Groups - Failure analysis

Failure scenario

A team keys events randomly for throughput but later needs per-user ordering; related events are spread across partitions.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Kafka: Logs, Partitions, and Consumer Groups - Build, inspect, and read deeper

Small implementation / lab

Run Kafka/Redpanda locally with three partitions; produce keyed messages, add/remove consumers, and observe rebalances and offsets.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Apache Kafka design](https://kafka.apache.org/design/) - <https://kafka.apache.org/design/>

[Apache Kafka docs](https://kafka.apache.org/documentation/) - <https://kafka.apache.org/documentation/>

Recall checklist

- Why does Kafka exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 30 - Delivery Semantics and Idempotent Consumers

Why this exists. Networks can lose acknowledgements even when work succeeds. Reliable consumers therefore assume duplicates are possible and make side effects idempotent or deduplicated.

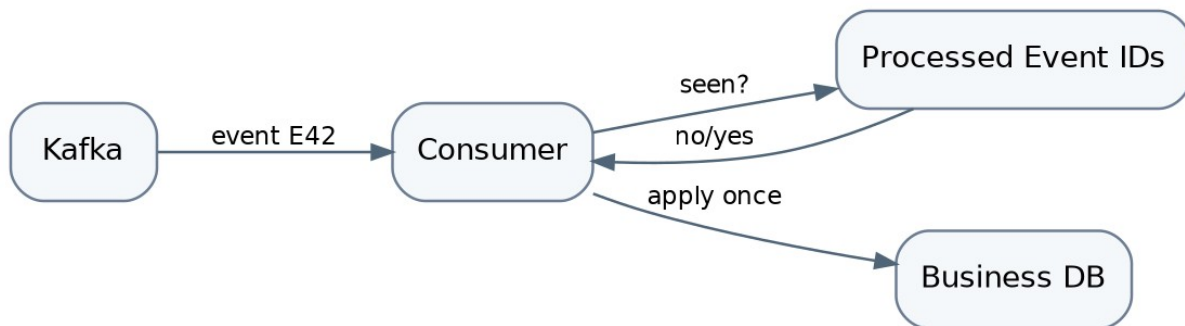


Figure 30.1 - Delivery Semantics and Idempotent Consumers: simplified architecture

Three concepts to retain

- At-most-once may lose work; at-least-once may duplicate work.
- Exactly-once is scoped and usually depends on transactional boundaries between log and state.
- Business idempotency keys are often the pragmatic solution.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Delivery Semantics and Idempotent Consumers - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Duplicate scenario

Consumer writes DB successfully, crashes before committing offset, restarts, and processes the same event again.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If delivery semantics becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Delivery Semantics and Idempotent Consumers - Failure analysis

Failure scenario

A payment-event consumer sends two emails and double-applies loyalty points after a crash/replay.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Delivery Semantics and Idempotent Consumers - Build, inspect, and read deeper

Small implementation / lab

Implement an inbox/dedup table keyed by event_id; replay the same Kafka event repeatedly and prove one business effect.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Kafka design - delivery semantics](https://kafka.apache.org/design/) - <https://kafka.apache.org/design/>

[Stripe idempotency](https://docs.stripe.com/api/idempotent_requests) - https://docs.stripe.com/api/idempotent_requests

Recall checklist

- Why does delivery semantics exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 31 - Retry Topics, Poison Messages, and DLQs

Why this exists. Retries must distinguish transient failures from permanent bad messages. Immediate retry loops amplify incidents; delayed retry topics and DLQs isolate failures from live traffic.

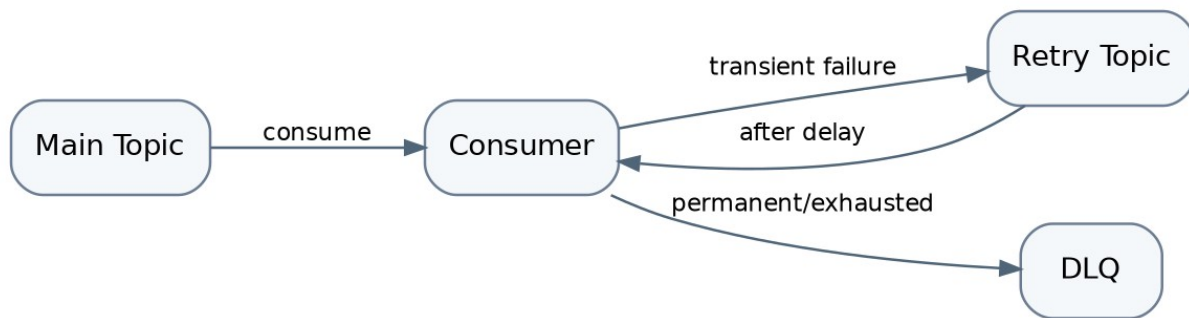


Figure 31.1 - Retry Topics, Poison Messages, and DLQs: simplified architecture

Three concepts to retain

- Exponential backoff increases time between attempts.
- Retry topics move failed events out of the hot path.
- DLQs need operational tooling for inspect, purge, repair, and replay.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Retry Topics, Poison Messages, and DLQs - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Amplification

If every failed request retries three times during a dependency outage, a 1k RPS workload can become roughly 4k attempted operations.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If retries DLQ becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Retry Topics, Poison Messages, and DLQs - Failure analysis

Failure scenario

One malformed event repeatedly crashes a consumer at the same offset and blocks all later events in that partition.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Retry Topics, Poison Messages, and DLQs - Build, inspect, and read deeper

Small implementation / lab

Create main, retry-1m, retry-10m, and DLQ topics; inject transient and permanent failures and inspect event history.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Uber reliable reprocessing](https://www.uber.com/in/en/blog/reliable-reprocessing/) - <https://www.uber.com/in/en/blog/reliable-reprocessing/>

Recall checklist

- Why does retries DLQ exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 32 - Transactional Outbox and the Dual-Write Problem

Why this exists. Writing business state and publishing an event are two separate systems. Without an atomic boundary, one can succeed while the other fails, producing missing or phantom events.

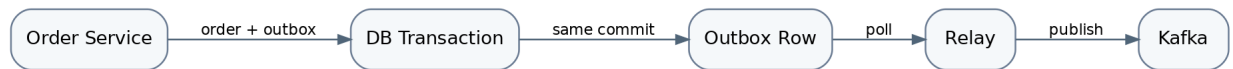


Figure 32.1 - Transactional Outbox and the Dual-Write Problem: simplified architecture

Three concepts to retain

- Write business row and outbox row in one database transaction.
- A relay publishes outbox rows to the broker asynchronously.
- Consumers still need idempotency because relay delivery may repeat.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Transactional Outbox and the Dual-Write Problem - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Dual-write failure

DB COMMIT succeeds; Kafka publish times out permanently. The order exists but downstream systems never hear about it.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If transactional outbox becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Transactional Outbox and the Dual-Write Problem - Failure analysis

Failure scenario

A service publishes “OrderCreated” first, then DB insert fails; downstream ships an order that does not exist.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Transactional Outbox and the Dual-Write Problem - Build, inspect, and read deeper

Small implementation / lab

Implement an orders + outbox table and a polling relay; kill the relay between publish and marking sent to observe duplicates.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Microservices.io transactional outbox](https://microservices.io/patterns/data/transactional-outbox.html) - <https://microservices.io/patterns/data/transactional-outbox.html>

Recall checklist

- Why does transactional outbox exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 33 - Sagas and Distributed Business Transactions

Why this exists. A saga coordinates a business process across services through local transactions and compensating actions rather than a single global ACID transaction.

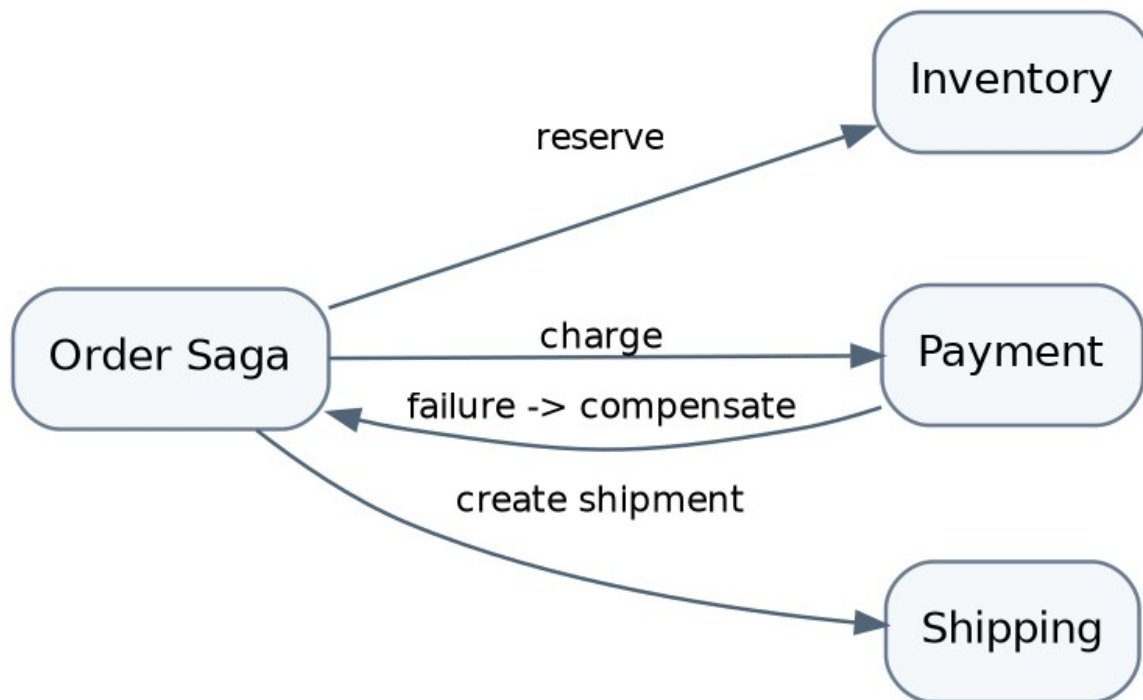


Figure 33.1 - Sagas and Distributed Business Transactions: simplified architecture

Three concepts to retain

- Orchestration centralizes workflow decisions; choreography reacts to events.
- Compensation is business logic, not database rollback.
- Some actions are not truly reversible and require semantic remediation.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Sagas and Distributed Business Transactions - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Order saga

Reserve inventory -> charge card -> arrange shipping; if shipping fails, refund and release inventory.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If sagas becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Sagas and Distributed Business Transactions - Failure analysis

Failure scenario

A compensation assumes refund always succeeds, then payment provider remains unavailable and the system loses track of money owed.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Sagas and Distributed Business Transactions - Build, inspect, and read deeper

Small implementation / lab

Implement a three-step order saga with persisted state and deterministic failure injection at every transition.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Microsoft Saga pattern](https://learn.microsoft.com/en-us/azure/architecture/reference-architectures/saga/saga) - <https://learn.microsoft.com/en-us/azure/architecture/reference-architectures/saga/saga>

Recall checklist

- Why does sagas exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 34 - Consistency Models and Convergence

Why this exists. Consistency is an application contract about what reads/writes may observe. Strong coordination simplifies reasoning; convergent systems accept temporary disagreement in exchange for availability and latency.



Figure 34.1 - Consistency Models and Convergence: simplified architecture

Three concepts to retain

- Linearizability makes operations appear atomic in real-time order.
- Session guarantees such as read-your-writes can be sufficient for many user experiences.
- CRDT-style structures encode deterministic merge behavior for concurrent updates.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Consistency Models and Convergence

- Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Invariant distinction

Replica convergence does not guarantee a business invariant such as “sell at most one remaining seat.”

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If consistency becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Consistency Models and Convergence

- Failure analysis

Failure scenario

Two disconnected regions each sell the final unit of inventory; replicas later converge perfectly to zero but the product was oversold.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Consistency Models and Convergence

- Build, inspect, and read deeper

Small implementation / lab

Implement a grow-only counter on two disconnected replicas and merge it; contrast with inventory where naive convergence is insufficient.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[ACM Queue - Research for Practice: Convergence](https://queue.acm.org/detail.cfm?id=3561801) - <https://queue.acm.org/detail.cfm?id=3561801>

Recall checklist

- Why does consistency exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

PART V - RELIABILITY AND PRODUCTION OPERATIONS

The goal of this part is to build a reusable mental model, not memorize a product stack. Every mechanism is tied to an invariant, a bottleneck, or a failure mode.

Chapter 35 - Observability: Metrics, Logs, Traces, and Baggage

Why this exists. Monitoring tells you known conditions; observability helps explain unexpected behavior. Production systems need correlated metrics, structured logs, and traces linked by stable identities.

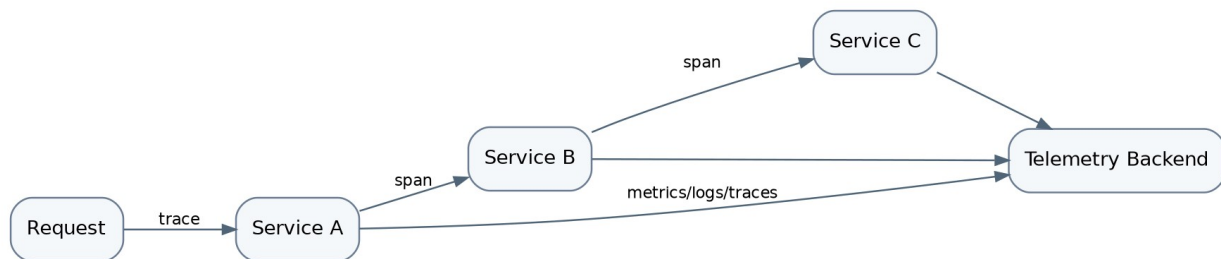


Figure 35.1 - Observability: Metrics, Logs, Traces, and Baggage: simplified architecture

Three concepts to retain

- Metrics summarize populations efficiently.
- Logs capture detailed events; structured logs make querying reliable.
- Traces reconstruct a request path across distributed services.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Observability: Metrics, Logs, Traces, and Baggage - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Signal use

Use metrics to detect, traces to localize, logs to explain.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If observability becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Observability: Metrics, Logs, Traces, and Baggage - Failure analysis

Failure scenario

Every service reports healthy CPU and 200s, yet a subset of requests takes 8 seconds because one downstream branch is slow; without traces the path is invisible.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Observability: Metrics, Logs, Traces, and Baggage - Build, inspect, and read deeper

Small implementation / lab

Instrument a three-service request with OpenTelemetry; correlate trace_id into structured logs and compare p95 span durations.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[OpenTelemetry signals](https://opentelemetry.io/docs/concepts/signals/) - <https://opentelemetry.io/docs/concepts/signals/>

[Google Dapper](https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/) - <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>

[Stripe canonical logs](https://stripe.com/blog/canonical-log-lines) - <https://stripe.com/blog/canonical-log-lines>

Recall checklist

- Why does observability exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 36 - SLIs, SLOs, Error Budgets, and Burn Rate

Why this exists. SLOs convert “reliable enough” into a measurable contract. Error budgets make reliability an explicit trade-off; burn rate determines how urgently current failures threaten that budget.

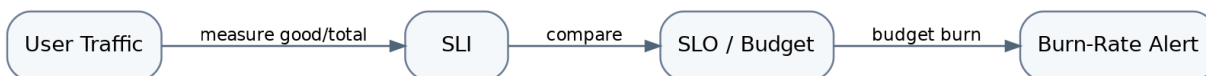


Figure 36.1 - SLIs, SLOs, Error Budgets, and Burn Rate: simplified architecture

Three concepts to retain

- SLIs measure user-visible good events over valid events.
- SLOs define acceptable reliability over a time window.
- Multi-window burn-rate alerts detect both fast outages and slow persistent degradation.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

SLIs, SLOs, Error Budgets, and Burn Rate - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Burn rate

For 99.9% SLO, allowed error rate is 0.1%. Observed 1.4% errors means 14x burn rate.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If SRE becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

SLIs, SLOs, Error Budgets, and Burn Rate - Failure analysis

Failure scenario

A team paged on CPU >80% even while users are fine, but misses a severe correctness failure that returns HTTP 200.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

SLIs, SLOs, Error Budgets, and Burn Rate - Build, inspect, and read deeper

Small implementation / lab

Generate synthetic request data, define good-event criteria, calculate 28-day error budget and fast/slow burn alerts.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Google SRE - Alerting on SLOs](https://sre.google/workbook/alerting-on-slos/) - <https://sre.google/workbook/alerting-on-slos/>

[Google SRE - Monitoring distributed systems](https://sre.google/sre-book/monitoring-distributed-systems/) - <https://sre.google/sre-book/monitoring-distributed-systems/>

Recall checklist

- Why does SRE exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 37 - Timeouts, Retries, Backoff, and Jitter

Why this exists. A retry is extra load sent to a system that may already be unhealthy. Reliable clients bound retry count, propagate deadlines, back off, add jitter, and retry only potentially transient failures.



Figure 37.1 - Timeouts, Retries, Backoff, and Jitter: simplified architecture

Three concepts to retain

- Timeouts bound one operation; deadlines bound the whole request.
- Exponential backoff spaces repeated attempts.
- Jitter prevents synchronized clients from retrying at the same moments.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Timeouts, Retries, Backoff, and Jitter

- Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Deadline

A 2-second end-to-end deadline cannot safely contain three independent 2-second dependency timeouts.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If retries becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Timeouts, Retries, Backoff, and Jitter

- Failure analysis

Failure scenario

Thousands of clients retry every 1 second after a regional dependency failure, creating synchronized traffic spikes that prevent recovery.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Timeouts, Retries, Backoff, and Jitter

- Build, inspect, and read deeper

Small implementation / lab

Simulate 1000 clients with fixed retry vs exponential backoff+jitter; compare peak retry traffic.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[AWS Builders Library - Timeouts, retries, and backoff with jitter](https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/) -

<https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>

Recall checklist

- Why does retries exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 38 - Circuit Breakers, Bulkheads, and Dependency Isolation

Why this exists. When a dependency is failing, continuing to send all traffic can waste threads, queue capacity, and time. Circuit breakers and bulkheads limit how far one dependency failure propagates.

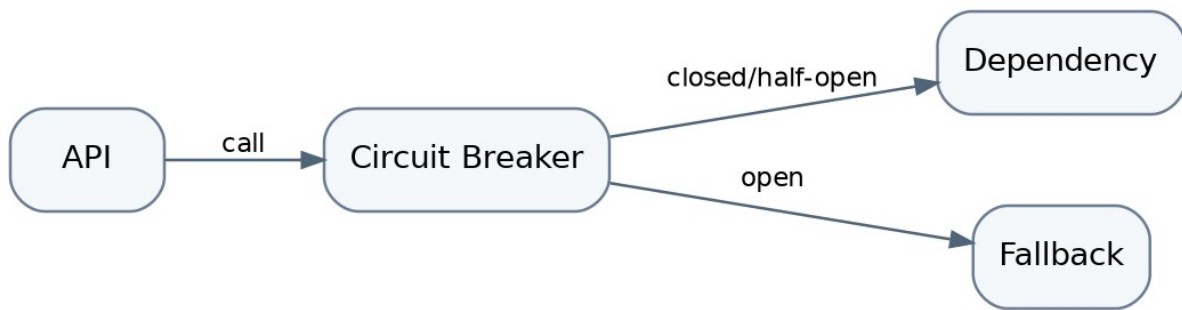


Figure 38.1 - Circuit Breakers, Bulkheads, and Dependency Isolation: simplified architecture

Three concepts to retain

- Circuit breakers move from closed to open to half-open based on failures/recovery probes.
- Bulkheads reserve independent capacity so one workload cannot consume every worker.
- Fallbacks must be semantically safe, not merely available.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Circuit Breakers, Bulkheads, and Dependency Isolation - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Isolation

If search exhausts the same connection pool used by checkout, a search incident can take down revenue.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If resilience patterns becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Circuit Breakers, Bulkheads, and Dependency Isolation - Failure analysis

Failure scenario

Circuit breaker opens globally for a multi-tenant dependency after one tenant sends malformed requests; healthy tenants are unnecessarily blocked.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Circuit Breakers, Bulkheads, and Dependency Isolation - Build, inspect, and read deeper

Small implementation / lab

Implement a breaker around an unreliable endpoint and two independent worker pools for critical vs background traffic.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Microsoft circuit breaker pattern](https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker) - <https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker>

Recall checklist

- Why does resilience patterns exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 39 - Load Shedding and Admission Control

Why this exists. A healthy overloaded system rejects some work early so critical work can still succeed. Accepting everything can reduce successful throughput by causing queue growth, timeouts, and retries.



Figure 39.1 - Load Shedding and Admission Control: simplified architecture

Three concepts to retain

- Admission control protects capacity before work enters expensive stages.
- Priority classes preserve high-value traffic during incidents.
- Concurrency limits often protect scarce resources better than raw RPS limits.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Load Shedding and Admission Control - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Counterintuitive throughput

Rejecting 500 of 1500 RPS can keep a 1000 RPS service healthy; accepting all 1500 may cause far fewer than 1000 successes after retry amplification.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If load shedding becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Load Shedding and Admission Control - Failure analysis

Failure scenario

Background exports consume every worker, causing interactive checkout to time out even though total fleet CPU appears moderate.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Load Shedding and Admission Control - Build, inspect, and read deeper

Small implementation / lab

Create critical and background request classes with separate concurrency budgets; overload both and verify critical p95 remains bounded.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Stripe rate limiters](https://stripe.com/blog/rate-limiters) - <https://stripe.com/blog/rate-limiters>

[Google SRE cascading failures](https://sre.google/sre-book/addressing-cascading-failures/) - <https://sre.google/sre-book/addressing-cascading-failures/>

Recall checklist

- Why does load shedding exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 40 - Canaries, Feature Flags, and Rollback

Why this exists. Deployment changes risk by introducing new code to production. Canary cohorts and feature flags bound blast radius, while rollback requires data/schema compatibility rather than just restoring the old binary.

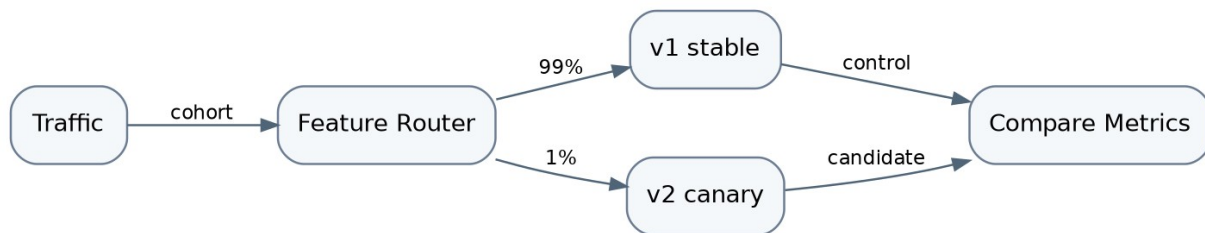


Figure 40.1 - Canaries, Feature Flags, and Rollback: simplified architecture

Three concepts to retain

- Deploying code and releasing behavior can be separate operations.
- Stable cohorts make candidate vs control comparisons meaningful.
- Backward-compatible migrations keep old and new versions able to coexist.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Canaries, Feature Flags, and Rollback - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Risk model

Operational risk is roughly probability of regression multiplied by blast radius.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If deployments becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Canaries, Feature Flags, and Rollback - Failure analysis

Failure scenario

v2 writes a new incompatible schema, then a rollback starts v1 binaries that cannot read those rows.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Canaries, Feature Flags, and Rollback

- Build, inspect, and read deeper

Small implementation / lab

Implement deterministic cohort routing by `hash(user_id)%100` and automatic rollback on candidate error/latency thresholds.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Martin Fowler - Feature Toggles](https://martinfowler.com/articles/feature-toggles.html) - <https://martinfowler.com/articles/feature-toggles.html>

Recall checklist

- Why does deployments exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 41 - Multi-Region Design and Disaster Recovery

Why this exists. Multi-region systems must define which failures they survive, where writes are authoritative, how data replicates, and what RPO/RTO is acceptable. Active-active is not free availability.

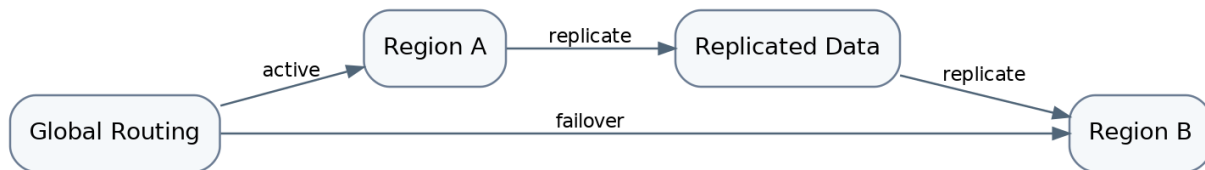


Figure 41.1 - Multi-Region Design and Disaster Recovery: simplified architecture

Three concepts to retain

- RPO describes tolerable data loss; RTO describes acceptable recovery time.
- Active-passive is simpler but may have slower failover.
- Active-active requires conflict, routing, and consistency decisions for concurrent regional work.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Multi-Region Design and Disaster Recovery - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

RPO/RTO

RPO=5 min means up to five minutes of committed data may be lost; RTO=30 min means service recovery may take half an hour.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If multi-region becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Multi-Region Design and Disaster Recovery - Failure analysis

Failure scenario

DNS fails users to region B, but database writes are still authoritative only in region A and replication is minutes behind.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Multi-Region Design and Disaster Recovery - Build, inspect, and read deeper

Small implementation / lab

Design DR for a billing API with stated RPO/RTO; rehearse losing one region and list exact failover steps and data risks.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[Google Cloud disaster recovery planning](https://cloud.google.com/architecture/dr-scenarios-planning-guide) - <https://cloud.google.com/architecture/dr-scenarios-planning-guide>

Recall checklist

- Why does multi-region exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

Chapter 42 - Schema and Data Migrations Without Downtime

Why this exists. Production data migrations are multi-version distributed-system changes. Safe migrations separate additive schema change, backfill, dual-read/write, cutover, and cleanup.

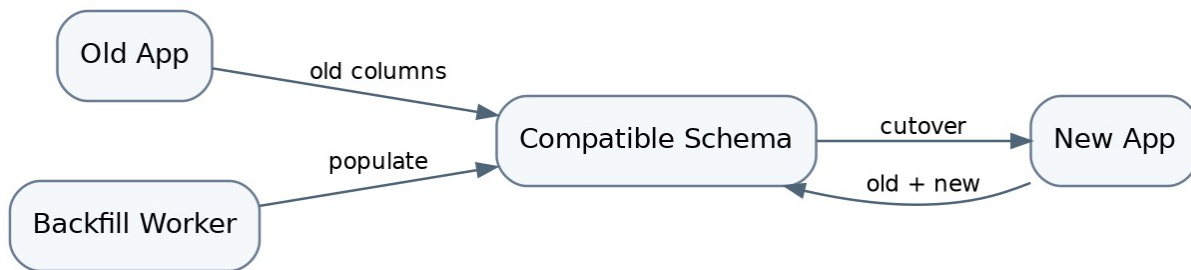


Figure 42.1 - Schema and Data Migrations Without Downtime: simplified architecture

Three concepts to retain

- Expand-contract keeps old/new code compatible during rollout.
- Backfills must be resumable, rate-limited, and observable.
- Shadow reads and comparison detect semantic differences before cutover.

Senior engineer lens

Identify the invariant, the bottleneck, and the failure boundary before choosing technology. The same product can legitimately use different mechanisms for different data paths.

Schema and Data Migrations Without Downtime - Mechanics and worked example

Mechanics

Start from the request or state transition, then identify which component owns each decision. Be explicit about synchronous boundaries: every synchronous hop extends the critical path and can propagate failure upstream. Be equally explicit about state: local memory, shared cache, replicated database, queue, and object store have different durability and consistency contracts.

A robust design also defines identifiers and replay behavior. Request IDs, event IDs, shard keys, transaction IDs, trace IDs, and idempotency keys are not bookkeeping details; they are the handles that let the system detect duplicates, correlate work, route state, and recover safely.

Migration sequence

Add nullable column -> deploy dual write -> backfill -> switch reads -> enforce constraint -> remove old column.

Trade-offs to reason through

- Latency vs coordination
- Availability vs consistency
- Throughput vs per-request isolation
- Operational simplicity vs horizontal scale
- Freshness vs caching
- Cost vs headroom

Design-review prompt

If migrations becomes the dominant bottleneck at 10x traffic, which metric would reveal it first, and what change would increase capacity without silently weakening correctness?

Schema and Data Migrations Without Downtime - Failure analysis

Failure scenario

A backfill scans the primary at full speed, saturates I/O, and causes customer traffic latency to spike.

Reason through the incident

1. What fails first?
2. What user-visible symptom appears?
3. Which retries or queues amplify the problem?
4. What stale or partial state can appear?
5. What telemetry proves the hypothesis?
6. What should degrade, reject, or fail over?
7. How do you recover without making the incident worse?

Failure-path mental model

Do not stop at “component X is down.” Ask how callers behave while it is down. Timeouts occupy resources. Retries add traffic. Queues turn overload into stored waiting. Caches can convert failure into database load. Failover can expose stale replicas. A local defect becomes an outage through these feedback loops.

Invariant check

Write one sentence that must remain true during this failure. Examples: “A payment is never charged twice,” “committed file metadata never points to a nonexistent blob,” or “critical checkout traffic retains capacity even if analytics is shed.”

Schema and Data Migrations Without Downtime - Build, inspect, and read deeper

Small implementation / lab

Perform an expand-contract migration on a local Postgres table while continuously serving reads/writes from two app versions.

What to measure

- correctness/invariant violations
- throughput
- p50/p95/p99 latency
- error rate
- saturation/queue depth
- recovery time after injected failure

Official and primary references

[GitHub database partitioning](https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/) - <https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/>

[PostgreSQL ALTER TABLE](https://www.postgresql.org/docs/current/sql-altertable.html) - <https://www.postgresql.org/docs/current/sql-altertable.html>

Recall checklist

- Why does migrations exist?
- What does the simplest correct architecture look like?
- Which failure mode is most dangerous?
- Which metric tells you it is saturating?
- What would you build to prove you understand it?

PART VI - ARCHITECTURE AUTOPSIES

Real systems are useful when treated as evidence, not templates. For each case, separate the company-specific technology choice from the general design principle that transfers to your own systems.

Chapter 43 - Discord: Storing Trillions of Messages

The production problem

Discord moved from MongoDB to Cassandra and later to ScyllaDB as operational toil and tail latency became unacceptable. The critical design lesson is not “use ScyllaDB”; it is to connect partition-key design, workload skew, operational cost, and migration strategy.



Architecture lesson

Channel ID + time bucket preserves locality while bounding partition size. Snowflake IDs provide sortable message identity. Rust-based data services isolate clients from database-specific behavior.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Discord: Storing Trillions of Messages

- Failure and operations

Failure / stress case

During the World Cup, a few huge communities created extreme message spikes; a good partition model and a storage engine with predictable tail behavior mattered more than average throughput.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Recreate a message table partitioned by channel and month, generate a Zipf-distributed workload, then identify hot channels and test alternate bucket widths.

Discord: Storing Trillions of Messages

- Source notes and transfer

Primary engineering source

[Discord - How Discord Stores Trillions of Messages](https://discord.com/blog/how-discord-stores-trillions-of-messages) - <https://discord.com/blog/how-discord-stores-trillions-of-messages>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 44 - Discord: Indexing Trillions of Messages

The production problem

Discord evolved search infrastructure when very large Elasticsearch clusters became hard to upgrade, restart, and shard safely. Search has its own data model, indexing pipeline, lag, backfill, and failure modes separate from source-of-truth storage.



Architecture lesson

Source data feeds an indexing queue; workers bulk-index into many manageable search clusters/indices. Large guilds require partition strategies that avoid Lucene document-count ceilings and operationally huge shards.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Discord: Indexing Trillions of Messages - Failure and operations

Failure / stress case

Search can be temporarily stale while source messages remain correct; that is a product-level consistency decision, not a database bug.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Build Postgres -> queue -> search-index pipeline; pause the indexer for five minutes, then measure lag, catch-up throughput, and user-visible staleness.

Discord: Indexing Trillions of Messages - Source notes and transfer

Primary engineering source

[Discord - How Discord Indexes Trillions of Messages](https://discord.com/blog/how-discord-indexes-trillions-of-messages) - <https://discord.com/blog/how-discord-indexes-trillions-of-messages>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

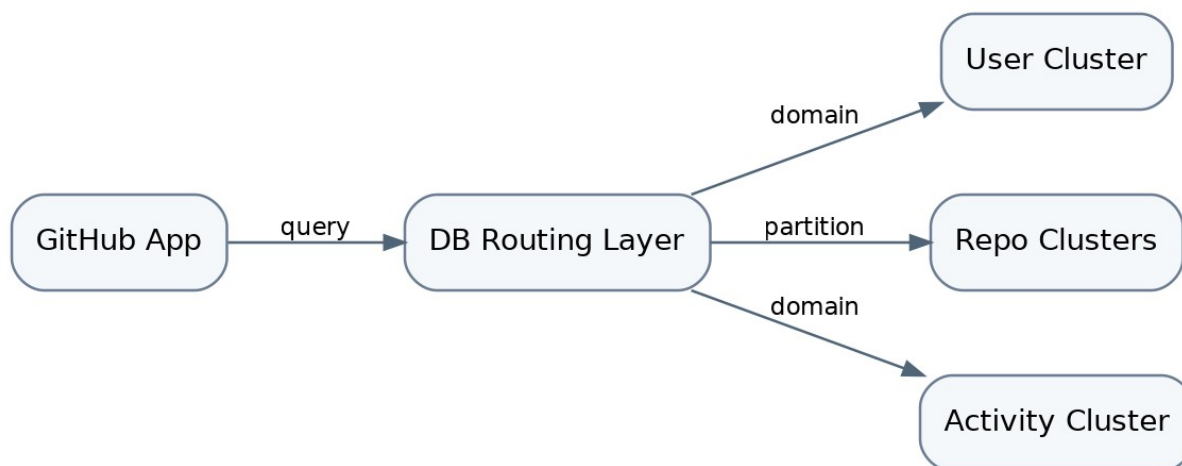
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 45 - GitHub: Partitioning Relational Databases

The production problem

GitHub kept relational semantics while progressively decomposing a central MySQL cluster. The case demonstrates vertical partitioning, read replicas, routing layers, and gradual operational migration rather than an overnight rewrite.



Architecture lesson

Start by moving domain tables to separate clusters, then introduce horizontal sharding where needed. Application tooling and linters must prevent accidental cross-partition access.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

GitHub: Partitioning Relational Databases - Failure and operations

Failure / stress case

Partitioning can reduce per-host load while making joins and transactions harder; the application architecture must make data ownership explicit.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Take a monolithic schema for users/repos/issues/PRs and propose vertical then horizontal partition boundaries; list every query that becomes cross-cluster.

GitHub: Partitioning Relational Databases - Source notes and transfer

Primary engineering source

[GitHub - Partitioning relational databases](https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/) - <https://github.blog/engineering/infrastructure/partitioning-githubs-relational-databases-scale/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

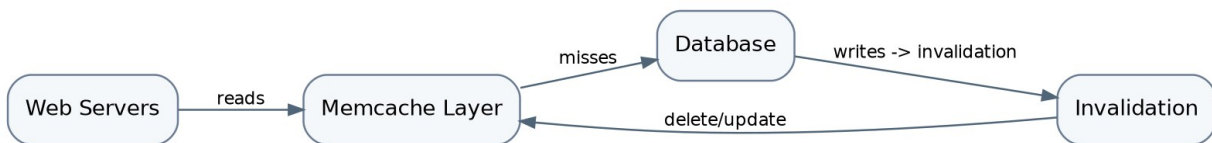
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 46 - Meta: Scaling Memcache

The production problem

Facebook/Meta used Memcache as a globally distributed cache layer with regional clusters, invalidation mechanisms, and careful handling of fan-out and hotspots. It demonstrates that caching at billion-user scale is a consistency and network problem, not just a hashmap.



Architecture lesson

Web servers read from cache first; misses reach backing stores. Invalidation propagate after writes. Regional layers reduce cross-region dependency and protect the database.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Meta: Scaling Memcache - Failure and operations

Failure / stress case

A cache cluster failure can shift enormous load to the database; overprovisioning, staged recovery, and miss-rate control are part of the design.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Simulate 1M keys with Zipf popularity; compare LRU hit rate, hot-key concentration, and database load during a cold-cache restart.

Meta: Scaling Memcache - Source notes and transfer

Primary engineering source

[Meta - Scaling Memcache at Facebook](https://engineering.fb.com/2013/04/15/core-infra/scaling-memcache-at-facebook/) - <https://engineering.fb.com/2013/04/15/core-infra/scaling-memcache-at-facebook/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

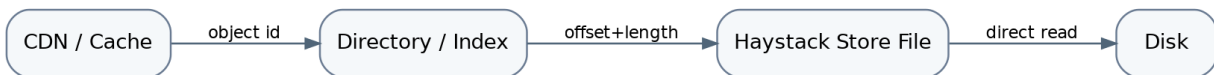
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 47 - Meta: Haystack Photo Storage

The production problem

Haystack was designed because storing billions of small image files made filesystem metadata operations a dominant cost. The system packs many objects into larger append-oriented files and uses a compact in-memory index for direct location.



Architecture lesson

Object ID maps to store file, offset, and length. Large sequential store files reduce expensive filesystem metadata lookups. CDN/cache layers serve hot content while Haystack handles durable origin reads.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Meta: Haystack Photo Storage - Failure and operations

Failure / stress case

The lesson generalizes: metadata can cost more than payload when object counts become enormous.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Build two stores for 100k small blobs: one-file-per-object and one append-only data file + SQLite offset index; compare inode count and random read overhead.

Meta: Haystack Photo Storage - Source notes and transfer

Primary engineering source

[Meta - Needle in a Haystack](https://engineering.fb.com/2009/04/30/core-infra/needle-in-a-haystack-efficient-storage-of-billions-of-photos/) - <https://engineering.fb.com/2009/04/30/core-infra/needle-in-a-haystack-efficient-storage-of-billions-of-photos/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

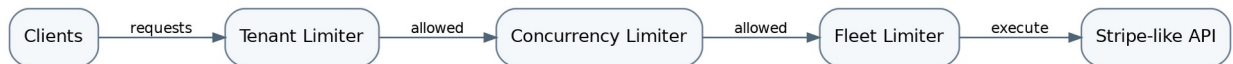
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 48 - Stripe: Rate Limiters as Reliability Infrastructure

The production problem

Stripe uses multiple limiters because “too much traffic” has several meanings: one user can spike QPS, one endpoint can consume concurrency, the whole fleet can saturate, or worker utilization can indicate overload.



Architecture lesson

Requests encounter limiters before expensive work. Different limiters protect tenant fairness, endpoint concurrency, and fleet capacity. Rejections are intentional load shedding.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Stripe: Rate Limiters as Reliability Infrastructure - Failure and operations

Failure / stress case

Correct 429/503 semantics and Retry-After behavior matter; bad clients can convert rejection into retry amplification.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Build a small API with per-user token bucket + global concurrency limiter + priority reservation; overload it and measure accepted p95 latency.

Stripe: Rate Limiters as Reliability Infrastructure - Source notes and transfer

Primary engineering source

[Stripe - Scaling your API with rate limiters](https://stripe.com/blog/rate-limiters) - <https://stripe.com/blog/rate-limiters>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

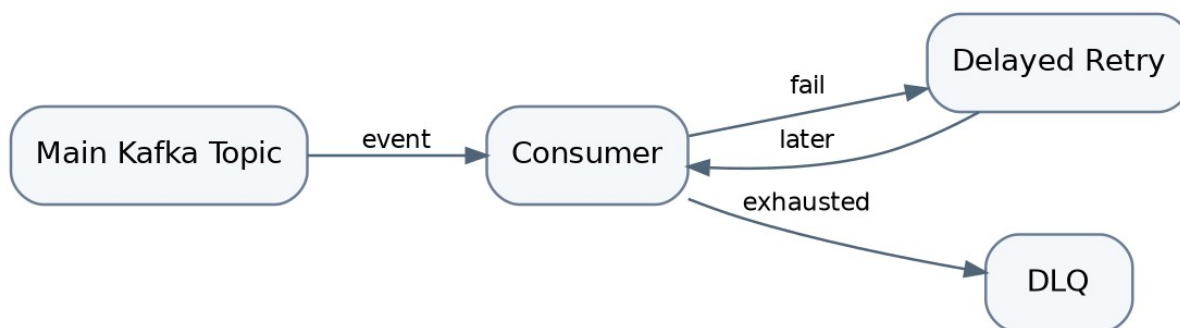
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 49 - Uber: Kafka Retry and Dead-Letter Architecture

The production problem

Uber separated failed processing from live traffic using retry topics with delays and a DLQ. This avoids head-of-line blocking and makes failure handling observable and operable.



Architecture lesson

A failed event moves to a delayed retry topic; after several tiers it lands in a DLQ. Operators need list/purge/replay workflows, not just storage.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Uber: Kafka Retry and Dead-Letter Architecture - Failure and operations

Failure / stress case

If every failure is retried immediately, a downstream outage becomes a self-generated traffic attack.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Implement main/retry/DLQ topics and a CLI that lists, patches, and replays DLQ events.

Uber: Kafka Retry and Dead-Letter Architecture - Source notes and transfer

Primary engineering source

[Uber - Reliable Reprocessing and DLQs](https://www.uber.com/in/en/blog/reliable-reprocessing/) - <https://www.uber.com/in/en/blog/reliable-reprocessing/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

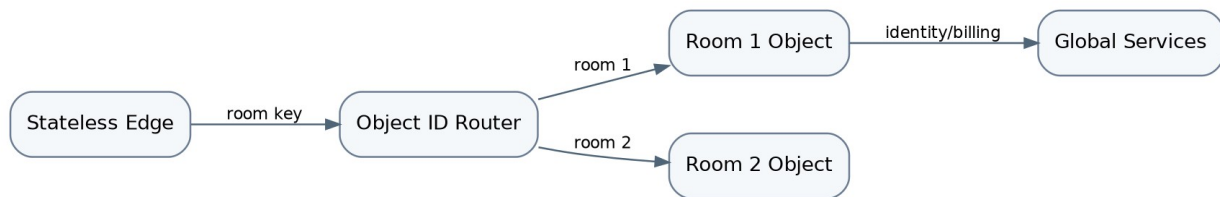
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 50 - Cloudflare: Durable Objects and Stateful Coordination

The production problem

Durable Objects place one globally unique stateful coordinator for a logical key such as a chat room, game, document, or rate limiter. The important idea is to align state scope with coordination scope.



Architecture lesson

Stateless edge workers route a key to one durable state owner. Each object serializes access to its own state, reducing distributed locking complexity within that key.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Cloudflare: Durable Objects and Stateful Coordination - Failure and operations

Failure / stress case

A single object that owns an entire global workload becomes a bottleneck; the object key must partition coordination appropriately.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Model collaborative study rooms as one object per room and define which state stays local versus global identity/billing/search services.

Cloudflare: Durable Objects and Stateful Coordination - Source notes and transfer

Primary engineering source

[Cloudflare - Rules of Durable Objects](https://developers.cloudflare.com/durable-objects/best-practices/rules-of-durable-objects/) - <https://developers.cloudflare.com/durable-objects/best-practices/rules-of-durable-objects/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 51 - Cloudflare: Consistent Hashing for Worker Warmth

The production problem

Cloudflare used a consistent hash ring to route low-traffic Workers to stable home servers, reducing duplicate warm instances and cold starts while minimizing disruption when servers join or leave.



Architecture lesson

Worker script IDs and servers map onto the same ring. Stable ownership preserves warm state; membership churn moves only nearby ranges.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Cloudflare: Consistent Hashing for Worker Warmth - Failure and operations

Failure / stress case

Affinity can conflict with load balance: a home server may be overloaded, so sharding requires refusal/load-shedding behavior and a fallback path.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Implement consistent hashing for 300 simulated servers and 50k workers; compare warm-hit rate and remapping under churn.

Cloudflare: Consistent Hashing for Worker Warmth - Source notes and transfer

Primary engineering source

[Cloudflare - Eliminating Cold Starts 2](https://blog.cloudflare.com/eliminating-cold-starts-2-shard-and-conquer/) - <https://blog.cloudflare.com/eliminating-cold-starts-2-shard-and-conquer/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

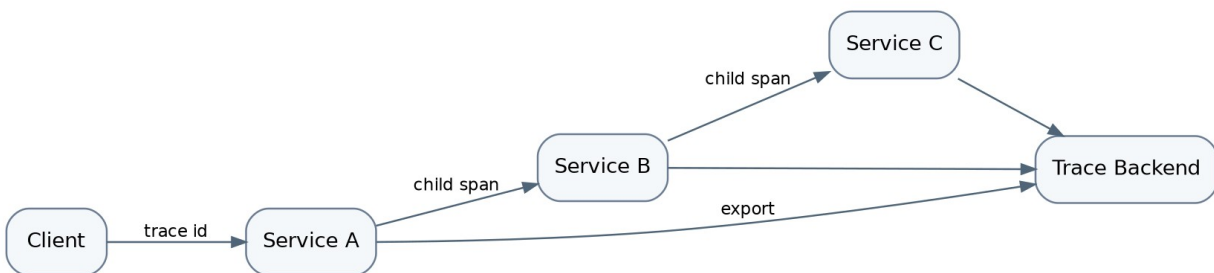
Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

Chapter 52 - Google: Dapper and Distributed Tracing

The production problem

Dapper demonstrated that low-overhead tracing can reconstruct requests across thousands of services. Modern OpenTelemetry follows the same core abstraction: traces contain spans connected by parent/child context.



Architecture lesson

Common RPC libraries propagate trace context. Sampling controls cost. The trace lets operators identify which dependency dominated latency or failed.

What to copy - and what not to copy

Copy the reasoning: workload shape, invariant, bottleneck, migration strategy, and operational control. Do not copy the company's database or queue merely because the scale story is impressive.

Google: Dapper and Distributed Tracing - Failure and operations

Failure / stress case

Sampling means traces are not a complete accounting system; high-cardinality metrics/logs and traces answer different questions.

Operational questions

- How is load distributed?
- Where can one hot tenant/key/shard dominate?
- How is bad state detected?
- Can the system be upgraded without full downtime?
- What is the rollback or replay path?
- Which component has the highest operational toil?

Rebuild the idea yourself

Instrument a five-service toy request, sample 10%, then create a latency incident in one leaf service and locate it from trace waterfalls.

Google: Dapper and Distributed Tracing - Source notes and transfer

Primary engineering source

[Google Research - Dapper](https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/) - <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>

[OpenTelemetry traces](https://opentelemetry.io/docs/concepts/signals/traces/) - <https://opentelemetry.io/docs/concepts/signals/traces/>

Transferable mental model

Translate the architecture into three statements: (1) what state/work was becoming expensive, (2) what partitioning/routing/coordination choice changed the cost curve, and (3) what new failure mode the redesign introduced. This translation is the part that should survive after you forget the exact vendor technology.

Architecture interview drill

Assume you inherit a smaller version of this system at 1/1000th the traffic. Which mechanisms would you keep, which would you remove, and what signals would tell you when to introduce them later?

PART VII - FIVE PROJECTS TO BECOME PRACTICALLY STRONG

These projects are intentionally from different system categories. The goal is not to finish five CRUD apps; it is to repeatedly design, measure, break, recover, and explain trade-offs.

Project 1 - URL Shortener at Scale

Build a production URL shortener that starts on one PostgreSQL instance and evolves to cached reads, generated IDs, abuse controls, partitioning, analytics events, and multi-region redirect serving.

Architecture milestones

- REST API + idempotent create
- Base62/Snowflake ID strategy
- PostgreSQL schema + indexes
- Redis cache-aside
- Click event Kafka pipeline
- Rate limits + abuse prevention
- Load test and SLO

Required experiment

Establish a baseline, run a load test, inject at least two failures, capture traces/metrics, then redesign one bottleneck. Write a one-page incident review explaining the before/after architecture and evidence.

Definition of done

You can draw the architecture from memory, calculate its expected capacity, explain its consistency model, demonstrate a controlled failure, and justify why each major component exists.

Project 1 - URL Shortener at Scale - Senior design review

1. What is the source of truth?
2. What is partitioned, and by what key?
3. Which path is synchronous?
4. What can be eventually consistent?
5. What happens at 10x traffic?
6. What happens when the queue or cache is unavailable?
7. How are duplicate requests/events handled?
8. What SLO would you publish?
9. How would you deploy v2 safely?
10. What would you remove if traffic were 100x smaller?

Project 2 - Discord/Slack-like Chat

Build a real-time chat backend with websocket gateways, presence, partitioned message storage, read receipts, notifications, search indexing, and retryable background jobs.

Architecture milestones

- WebSocket gateway
- Message IDs and channel partitioning
- Kafka events
- Presence state
- Search index lag
- Hot-channel handling
- DLQ + replay tooling

Required experiment

Establish a baseline, run a load test, inject at least two failures, capture traces/metrics, then redesign one bottleneck. Write a one-page incident review explaining the before/after architecture and evidence.

Definition of done

You can draw the architecture from memory, calculate its expected capacity, explain its consistency model, demonstrate a controlled failure, and justify why each major component exists.

Project 2 - Discord/Slack-like Chat - Senior design review

1. What is the source of truth?
2. What is partitioned, and by what key?
3. Which path is synchronous?
4. What can be eventually consistent?
5. What happens at 10x traffic?
6. What happens when the queue or cache is unavailable?
7. How are duplicate requests/events handled?
8. What SLO would you publish?
9. How would you deploy v2 safely?
10. What would you remove if traffic were 100x smaller?

Project 3 - YouTube-like Video Platform

Build video upload and delivery architecture with direct-to-object-storage upload, resumable chunks, transcoding queues, metadata DB, CDN distribution, and asynchronous moderation/analytics.

Architecture milestones

- Multipart upload
- Object storage
- Transcoding workers
- Queue/backpressure
- Metadata DB
- CDN cache policy
- Failure recovery

Required experiment

Establish a baseline, run a load test, inject at least two failures, capture traces/metrics, then redesign one bottleneck. Write a one-page incident review explaining the before/after architecture and evidence.

Definition of done

You can draw the architecture from memory, calculate its expected capacity, explain its consistency model, demonstrate a controlled failure, and justify why each major component exists.

Project 3 - YouTube-like Video Platform - Senior design review

1. What is the source of truth?
2. What is partitioned, and by what key?
3. Which path is synchronous?
4. What can be eventually consistent?
5. What happens at 10x traffic?
6. What happens when the queue or cache is unavailable?
7. How are duplicate requests/events handled?
8. What SLO would you publish?
9. How would you deploy v2 safely?
10. What would you remove if traffic were 100x smaller?

Project 4 - Google Drive/Dropbox Clone

Build a file-sync system with chunking, content hashes, delta upload, object storage, version metadata, notifications, offline reconciliation, and conflict handling.

Architecture milestones

- Chunk manifest
- Content-addressed blocks
- Version graph
- Strong metadata transactions
- Long-poll/WebSocket notification
- Offline sync
- Recovery from partial upload

Required experiment

Establish a baseline, run a load test, inject at least two failures, capture traces/metrics, then redesign one bottleneck. Write a one-page incident review explaining the before/after architecture and evidence.

Definition of done

You can draw the architecture from memory, calculate its expected capacity, explain its consistency model, demonstrate a controlled failure, and justify why each major component exists.

Project 4 - Google Drive/Dropbox Clone - Senior design review

1. What is the source of truth?
2. What is partitioned, and by what key?
3. Which path is synchronous?
4. What can be eventually consistent?
5. What happens at 10x traffic?
6. What happens when the queue or cache is unavailable?
7. How are duplicate requests/events handled?
8. What SLO would you publish?
9. How would you deploy v2 safely?
10. What would you remove if traffic were 100x smaller?

Project 5 - Large AI Learning Platform

Combine both books: collaborative rooms, PDF uploads, AI jobs, vector indexing, billing, notifications, analytics, Kafka, Postgres, Redis, object storage, SLOs, tracing, canaries, and overload controls.

Architecture milestones

- Capacity estimation
- API gateway + auth
- Postgres + Redis + object storage
- Kafka event backbone
- Search/vector indexes
- Durable AI jobs
- SLO/error budgets
- Canary releases + DR

Required experiment

Establish a baseline, run a load test, inject at least two failures, capture traces/metrics, then redesign one bottleneck. Write a one-page incident review explaining the before/after architecture and evidence.

Definition of done

You can draw the architecture from memory, calculate its expected capacity, explain its consistency model, demonstrate a controlled failure, and justify why each major component exists.

Project 5 - Large AI Learning Platform

- Senior design review

1. What is the source of truth?
2. What is partitioned, and by what key?
3. Which path is synchronous?
4. What can be eventually consistent?
5. What happens at 10x traffic?
6. What happens when the queue or cache is unavailable?
7. How are duplicate requests/events handled?
8. What SLO would you publish?
9. How would you deploy v2 safely?
10. What would you remove if traffic were 100x smaller?

Appendix A - System Design Interview / Design-Review Framework

- 1. Clarify functional requirements and non-functional constraints.
- 2. Estimate traffic, storage, bandwidth, concurrency, and growth.
- 3. Define external API and important data entities.
- 4. Draw the minimal end-to-end request path.
- 5. Identify source of truth and consistency requirements.
- 6. Add scaling mechanisms only where estimates demand them.
- 7. Define failure behavior, retry policy, overload controls, and recovery.
- 8. Define telemetry and SLOs.
- 9. Explain migration/deployment and rollback.
- 10. State explicit trade-offs and rejected alternatives.

Appendix B - Equations Worth Memorizing

- $L = \lambda * W$ (Little's Law).
- Backlog growth = $\lambda - \mu$ when $\lambda > \mu$.
- Average QPS = requests/day / 86,400.
- Peak QPS = average QPS * peak factor.
- Error budget = $1 - \text{SLO}$.
- Burn rate = observed error rate / allowed error rate.
- Quorum overlap heuristic: $R + W > N$.
- Independent fan-out: $P(\text{all within threshold}) = p^n$.
- Risk $\approx P(\text{failure}) * \text{blast radius}$.

Appendix C - Failure Injection Checklist

- Kill one API replica.
- Pause one Kafka consumer.
- Introduce 500 ms DB latency.
- Make cache unavailable.
- Expire a hot key simultaneously.
- Drop 10% of dependency calls.
- Return 429/503 from an external service.
- Create one poison event.
- Promote a lagging replica in staging.
- Deploy an intentionally slower canary.
- Fill a worker pool with background traffic.
- Delay indexing by five minutes.

Appendix D - Production Readiness Checklist

- Ownership and on-call are defined.
- SLIs/SLOs exist for user journeys.
- Timeouts and deadlines are explicit.
- Retries are bounded and jittered.
- Side effects are idempotent.
- Queues have backlog/age alerts and DLQ policy.
- Caches have miss-storm protection.
- Database backups and restore are tested.
- Schema changes are backward compatible.
- Canary + rollback are practiced.
- Metrics/logs/traces correlate by stable IDs.
- Capacity and load-shedding thresholds are documented.

Appendix E - Glossary

- Backpressure - signaling or enforcing that producers slow down when consumers cannot keep up.
- Burn rate - speed at which an SLO error budget is being consumed.
- Consensus - agreement on an ordered replicated state/log despite failures.
- Idempotency - repeated execution has the same business effect as one execution.
- Linearizability - operations appear atomic and respect real-time ordering.
- Partition - a subset of data/work assigned to an independent ownership boundary.
- RPO - maximum acceptable data-loss window.
- RTO - maximum acceptable recovery duration.
- Saturation - resource demand approaching/exceeding available capacity.
- Tail latency - high-percentile latency such as p95/p99, often dominated by stragglers.

Appendix F - Primary Source Index

PostgreSQL: <https://www.postgresql.org/docs/current/>

Kafka: <https://kafka.apache.org/documentation/>

Redis: <https://redis.io/docs/latest/>

OpenTelemetry: <https://opentelemetry.io/docs/>

Google SRE: <https://sre.google/>

Raft: <https://raft.github.io/>

Stripe Engineering: <https://stripe.com/blog/engineering>

GitHub Engineering: <https://github.blog/engineering/>

Discord Engineering: <https://discord.com/category/engineering>

Meta Engineering: <https://engineering.fb.com/>

Uber Engineering: <https://www.uber.com/blog/engineering/>

Cloudflare Engineering: <https://blog.cloudflare.com/tag/engineering/>

Final Recall Page - Design Systems, Not Diagrams

A strong system designer can start from a user journey, estimate the workload, choose where truth lives, define consistency, identify the saturation boundary, control overload, and explain how the system fails and recovers. Technology names are secondary. The durable skill is reasoning from constraints to trade-offs and validating those trade-offs with measurements.

The permanent questions

Where does truth live? What can be stale? What must never become false? What is the critical path? What happens at 10x traffic? What happens when one dependency disappears? How does overload propagate? How is the incident observed? How is a change rolled back?

End of Book 2 - Practical System Design